ISAF v2.0: From Instruction to Consequence

An Evidence-Bound Framework for AI Agent Governance, Authorization, Delegation and Consequential Action

Subodh KC | Research Lead and Lead Author

HAIEC Research

Final author-issued research paper · Version 2.0 · 9 October 2026 · Not peer reviewed · Contributor technical review invited

Keywords: agentic AI governance; AI agent governance; AI agent authorization; AI agent delegation; AI risk management; AI agent security; MCP security; AI auditability; AI decision rights; Evidence-Bound Assurance.

Contents

Abstract....05
Framework Overview....06

Part I: From Instruction to Consequence....08
1. The Stack Learned to Act....08
2. The All-Green Paradox....09
3. From AI Governance to Consequence Governance....10
4. A New Control Question: What Can This AI Actually Cause?....12
5. Contribution Statement....13

Part II: The Accountability Geometry....15
6. The Accountability Geometry....15
7. The Instruction Stack: What Shaped the Decision?....16
8. The Consequence Path: What Can the Decision Become?....17
9. The Five Evidence Planes: Five Different Truths....19
10. Delegation: The Right to Decide....20
11. Evidence Authority: What Is This Source Allowed to Prove?....22
12. Action Witness: Did It Actually Happen?....23
13. Effect Envelope: What Did the Action Change?....23
14. Evaluation Epoch: When Was This True?....24
15. The Proof Frontier: Where Does Evidence Stop?....24
16. Six Independent Closure Questions....26
17. The Value of the Geometry....27

Part III: Evidence That Earns Its Claims....27
18. The Harder Question: Why Should We Believe the Evidence?....27
19. Producer Qualification: How a Tool Earns Authority....28
20. Hard Negatives: Teaching the Framework What Not to Claim....29
21. Joern Qualification: Testing the Analyzer, Not Trusting the Tool....30
22. Kestrel: A Case Study in Knowing Where Proof Stops....31
23. Runtime Probes: Closing a Frontier Without Rewriting History....33
24. Guardrails: Configured Is Not Enforced....34
25. Telemetry: Observation Without Overclaiming....34
26. Recovery: Rollback Is Not the Same as Restoration....35
27. Change: What Is Different Since the Last Evaluation?....35
28. Forensics: Reconstruction Without Inventing Root Cause....36
29. From Evidence Collection to Evidence-Bound Assurance....37
30. Evidence-Bound Evaluation Method in Practice....37
31. Governance Interpretation....37

Part IV: Enterprise Operating Model for Trusted Autonomy....38
32. When Autonomy Becomes an Operating-Model Decision....38
33. Ownership: Who Owns the Agent?....38
34. Agent Identity: You Cannot Govern an Actor You Cannot Reliably Name....39
35. MCP and Tool Ecosystems: Authorization Is Necessary, Not Sufficient....40
36. Multi-Agent Systems: Handoff Is Not Delegation....41
37. Shared State: The Hidden Multi-Agent Control Plane....42
38. Human Approval: A Person in the Loop Is Not Yet a Control....42
39. Trusted Autonomy Is Risk-Adjusted Autonomy....43
40. The Board Does Not Need More AI Metrics. It Needs the Right Ones....44
41. The Operating Model Should Follow the Agent Lifecycle....44
42. Standards Should Interoperate: Not Collapse Into One Another....45
43. From AI Governance to a Digital Workforce Control Model....46
44. The End-to-End Scenario: The Autonomous Procurement Agent....46
45. Governance Without Slowing Innovation....48

46. Operating Principles.....48
Part V: Audit Protocol, Metrics, Crosswalk & Conclusion.....49
47. The Audit Should Begin With a Claim, Not a Tool.....49
48. The ISAF v2 Audit Protocol.....49
49. A Complete Example: The Role-Change Agent.....52
50. Checkpoint Architecture: Derive the Questions From the Model.....53
51. Metrics: Measure Assurance Without Rewarding False Certainty.....54
52. Falsification: How ISAF Should Try to Prove Itself Wrong.....55
52A. Final Applied Enterprise Findings: TM Forum Agentic Assurance 2026.....56
52B. Event Submission Provenance, Archived Reproduction and Reviewer Access: Fina….....57
52C. Archived Evidence Reproducibility and Contributor Technical Review Protocol.....59
53. Standards and Regulatory Crosswalk: Informative, Not Equivalence.....61
54. What ISAF v2 Does Not Claim.....63
55. Current Research Frontiers.....64
56. Recommended Assurance Output.....64
57. The Final Paradox.....64
58. Conclusion: Accountability Must Reach the Consequence.....65
References.....66

Research status and claim boundary

ISAF v2 is a research and assurance framework. HAIEC is a reference implementation of many of its concepts, not the normative definition of the framework. Product behavior, benchmark qualification, applied competition recognition and standards mappings are kept distinct throughout this paper. The framework does not claim peer review, universal safety proof, automatic compliance, or complete implementation of every research construct.

For Delegated Action Integrity (DCD), the empirical record is deliberately staged: frozen DCD-Bench v0.2 reference-suite conformance, post-hoc adversarial falsification of the unchanged evaluator, and a separately versioned hardened v0.3 evaluator that requalified against the frozen v0.2 baseline and the published failure properties. Independently curated held-out validation has not yet been executed.

Contributors & Advisory Credits

- Atul Mishra: Senior AI Architecture Advisor. Contributed enterprise AI architecture, agentic-system design, and telecom/enterprise use-case perspective; his background includes serving as AI Architect Lead for Johnson & Johnson and as TCS's North America AI Lead.
- Kushal Gautam: Data Engineering Advisor. Advised on data architecture, evidence flows, and data-engineering considerations relevant to the framework and implementation.
- Sameer Nepal: Public-Sector Perspective Advisor. Contributed government/public-sector operational, accountability, and governance perspective in an individual capacity; no institutional or government endorsement is implied.
- Abhishek Yadav: AI Research Contributor. Contributed graduate-level AI research support, literature review, and research assistance.
- Phurpa (Sameer) Thapa: Mathematical Modeling Advisor. Advised on mathematical-model framing and formalization.
- Roshan Basnet: Project Contributor. Provided general research, technical, and project support.
- Prajwal Urav: Project Contributor. Provided general research, technical, and project support.

Contribution note: These credits acknowledge advisory and supporting contributions. They do not imply co-authorship, institutional endorsement, or responsibility for every claim in the paper.

Acknowledged External Perspectives & Research Conversations

The following individuals contributed perspective through direct conversations, correspondence, research-review dialogue, or applied-domain discussion. They are acknowledged for perspective only; inclusion does not imply co-authorship, endorsement of ISAF v2, or responsibility for its claims.

- Dr. Brita Andercheck: Chief Data Officer, City of Dallas. Discussions on public-sector AI, data governance, agility, internal/sandbox AI use, and the operational realities of applying AI to city problems.
- Paul Challoner: Chief Technology Officer, Global Customer Unit AT&T, Ericsson. Discussions on AI security, telecom industrialization, autonomous systems, and the gap between declared boundaries and AI-reachable action paths.
- Sashieka Seneviratne: Director of Cloud RAN Solutions, AT&T Customer Unit, Ericsson. Conversations spanning telecom, AI, technology, entrepreneurship, and the practical context of Cloud RAN and network transformation.
- Yeshwant Muthusamy, Ph.D.: Founder/Principal, Yeshvik Solutions. Discussed HAIEC, enterprise AI assurance, governance, and enterprise adoption; briefing material was shared for feedback and follow-up dialogue.
- Adrian Salas: CISO, ShiftKey; board advisor. Discussed autonomous-system governance, authority, accountability, and evidence. Research material on the “Permission is not Delegation” direction was shared with him for requested feedback and industry perspective; no feedback or endorsement is attributed here unless separately documented.
- Dr. Thomas Edward Ainsworth: Founder and Principal Consultant, Human-Centered AI Governance Institute. Correspondence explored human authority, organizational accountability, governance, and the relationship between technical evidence and executive decision-making.
- Sriram Krishnan: Co-Founder & Chief Business Officer, Secufusion. A UTD Global Security Tour panelist with whom a subsequent in-person meeting was held; enterprise-security discussions considered runtime telemetry, AI detection/response, and the fit between runtime evidence and HAIEC’s source-backed action-assurance model.
- Nimesh Acharya: Banking-Domain External Perspective. Provided conversational-level input grounded in banking/financial-services operations and practical enterprise considerations. This is acknowledged as domain perspective only, not a formal research review, advisory role, or endorsement.
- Kevin Scott: Chief Technology Officer, PGA of America. Dallas AI discussion around PGA’s technology and AI journey provided enterprise transformation, operating-model, adoption, and execution perspective.
- Aamer Charania: Founding President & Chief AI Officer, Dallas AI. Dallas AI discussions provided broader enterprise-AI leadership, adoption, and ecosystem perspective.
- Babar Bhatti: Executive Vice President & Co-Founder, Dallas AI. Dallas AI discussions provided applied-AI, product, and enterprise-adoption perspective.

Abstract

AI governance was built for systems that generated outputs. Agentic AI changes the control problem because software can now choose, call tools, invoke APIs, delegate work, modify systems, and trigger business consequences. A control environment may successfully establish identity, permission, policy, and runtime activity while still leaving a harder question unresolved: was the consequential choice itself actually delegated to the agent?

ISAF v2.0 extends the Instruction Stack Audit Framework from instruction traceability to consequence-level assurance. The framework preserves the original nine-layer view of the forces that shape AI behavior, then follows the resulting decision through authority, access, reachable implementation paths, runtime evidence, effects, and consequences. It separates five evidence planes: Requested, Policy Authorized, Effectively Granted, Code Capable, and Observed, while treating delegated decision rights as an independent relation rather than another permission state (KC, 2025).

The framework is built around evidence discipline. A source may be strong for one claim and irrelevant for another. A valid credential can establish an effective grant; it cannot prove that a person delegated a business choice. Static code can establish capability; it cannot prove execution. A log can establish activity; it may not prove that an external state transition was applied. ISAF v2 therefore binds claims to source authority, provenance, scope, coverage, time, limitations, and an explicit proof frontier.

The central principle is simple: permission is not delegation. A system may be allowed to perform an operation without having been entrusted to choose every consequence-bearing parameter inside that operation. ISAF v2 makes those distinctions auditable so organizations can scale AI agents without allowing assurance claims to outrun the evidence that supports them.

Validation follows the same evidence discipline. DCD-Bench v0.2 matched all 20 frozen expected epistemic outcomes under sealed deterministic replay; a subsequent unchanged-evaluator red-team exposed 8/8 broader implementation-boundary failures; the separately versioned v0.3 hardened evaluator then passed 34/34 existing hardening tests while preserving all 20 v0.2 outcomes. No held-out cases or held-out gold were used, so the evidence supports bounded reference-evaluator qualification rather than production generalization. A separate 2026 TM Forum enterprise-pattern exercise supplies first-party applied control-test observations under frozen policy instances; those results have their own system/time/provenance and documented participant submission provenance and archived-source limitations (Sections 52A–52B), and they do not independently validate ISAF as a whole.

Framework Overview

The agent was authenticated. The credential was valid. The API call was allowed. The transaction was logged. Every control was green.

The decision was still not necessarily delegated.

ISAF v2 is designed around that control problem.

Enterprise AI is moving from systems that recommend to systems that act. The practical shift is larger than the phrase “agentic AI” suggests. When software begins selecting tools, choosing parameters, modifying state, or handing work to another agent, governance is no longer only a question of model behavior. It becomes a question of authority, decision rights, evidence, and consequence.

Organizations therefore face a new operating-model question: how do you scale autonomous capability without losing ownership, accountability, control effectiveness, or the ability to explain what happened? Security teams ask whether the agent is authenticated and authorized. Risk teams ask

whether controls exist. Developers ask whether the tool path works. Auditors ask for evidence. Boards ask who is accountable. These are related questions, but they are not the same question.

ISAF v2 provides one evidence-bound structure for keeping them separate.

First, it retains the original Instruction Stack: the layered sources that shape what an AI system does. Second, it adds a Consequence Path that follows a consequential action from request or influence through the agent, capability surface, runtime principal, handler, state transition, and resulting business effect. Third, it preserves five independent evidence planes: Requested, Policy Authorized, Effectively Granted, Code Capable, and Observed. Fourth, it overlays Delegation as a separate relation over unresolved consequential choices. Finally, it binds the entire evaluation to an Evidence Spine and an Evaluation Epoch so that source authority, provenance, coverage, time, limitations, and change remain visible.

The result is a different kind of AI agent audit. Instead of asking only whether a control is present, ISAF asks what the evidence actually establishes. Instead of turning missing evidence into a pass or a fail, it exposes the Proof Frontier, the exact point where the organization’s evidence stops.

The result is not a universal declaration that an AI system is “safe.” It is a bounded answer to more useful enterprise questions: What was requested? What did policy allow? What authority was actually granted? What could the code cause? What did qualified runtime evidence establish? Which consequential choices were delegated? Which controls were merely configured, and which were actually enforced? What changed between evaluations? What remains unknown?

A modern AI control environment can be all green and still be incomplete.

Part I: From Instruction to Consequence

1. The Stack Learned to Act

For most of the history of software governance, an instruction and an action were separated by people.

A system might recommend a loan decision, draft a message, flag a transaction, or predict equipment failure. A person, workflow, or deterministic service often sat between the recommendation and the consequence. The AI produced information. Something else acted.

That boundary is disappearing.

An AI agent can now choose a tool, call an API, browse a site, invoke an MCP server, write to a database, modify infrastructure, send an external message, create a ticket, issue a refund, schedule a job, delegate a task to another agent, or trigger a downstream workflow. The model's output is no longer necessarily the end of the system. It may be the beginning of an execution chain.

The object of governance changes with it.

The original Instruction Stack Audit Framework asked a foundational accountability question: what shaped the system's behavior? That question remains essential. ISAF v1 traced technical lineage across its historical Layers 0–8, from physical substrate through objective functions and training, with Layer 9 treating outputs and deployment separately. ISAF v2 preserves that lineage and adds explicit runtime instruction/influence provenance for system and developer instructions, retrieved context, memory, user input, tool context, and other sources that can shape a concrete agent decision.

But once the stack can act, traceability has to continue.

Suppose an enterprise agent is asked to "resolve the customer's complaint." The model determines that a refund is appropriate. It selects a refund tool. The tool is registered. The credential is valid. The payment service allows refunds up to a policy limit. The request succeeds.

Where did the consequential decision occur?

The obvious answer is the refund call. But the call contains several decisions. Which customer? Which transaction? Refund or replacement? What amount? Which destination? Immediately or after review? Was the amount copied from a trusted invoice, computed from a deterministic rule, selected from an approved range, or simply chosen by the model?

A tool name is not a decision-right model.

Tool inventory, permission checks, and logs remain necessary. They answer different questions. A tool inventory can show what exists. An authorization system can show what is permitted. IAM can show what a credential can access. Static analysis can show what code paths are reachable. Runtime telemetry can show what was observed. None of those facts, by itself, proves that an autonomous system was entrusted to choose a particular consequence-bearing value.

ISAF v2 therefore extends the audit horizontally.

Instruction / Influence → Agent / Orchestrator → Capability Surface → Runtime Principal & Scope → Handler / Operation → State Transition / Effect → Consequence

The vertical stack explains what shaped the decision. The horizontal path explains what the decision could become.

The framework calls this an instruction-to-consequence view because the governance problem now spans both.

That shift also changes the language of enterprise AI governance. The important questions are no longer limited to model quality, prompt controls, or access policies. They include ownership, delegated authority, decision rights, consequential reachability, control effectiveness, runtime evidence, and accountability for the resulting effect.

The stack learned to act. Accountability now has to follow it.

2. The All-Green Paradox

Consider a customer-support agent operating inside a well-managed enterprise environment.

The identity system recognizes the service principal. The OAuth token is valid. The refund API is listed in the approved toolset. Policy permits the service to issue refunds up to $500. The application calls the endpoint successfully. The runtime emits a trace. The transaction appears in the business system.

Now suppose the human instruction was only: “Resolve the customer’s complaint.”

The agent refunds $350.

Nothing in the control path necessarily failed.

The credential may have been valid. The policy may have permitted the action. The application may have been technically capable of executing it. The refund may really have happened.

And yet one question can remain unanswered:

Who gave the agent the right to decide $350?

This is the all-green paradox. Conventional controls can all succeed while decision-right provenance remains unresolved.

The distinction is easier to see when three authorization questions are separated.

Standing policy permission

Is the operation inside the approved policy envelope? A policy might say: this service may issue refunds up to $500.

Delegated decision rights

Was the autonomous system entrusted to resolve the consequence-bearing choice? A delegation might say: choose a refund amount up to $100 when documented shipping loss is established.

Per-action execution authorization

May this exact proposed action execute now? A runtime decision point might check the daily refund budget, current account state, required approvals, kill switches, fraud controls, or other mutable conditions.

These are not competing definitions of authorization. They are different predicates.

PolicyAuthorized(a) != DelegationComplete(a | K,D)

DelegationComplete(a | K,D) != ExecutionAuthorizedNow(a,s)

A mature AI control environment may need all three.

ISAF v2 preserves five independent evidence planes and deliberately keeps Delegation outside them. Requested, Policy Authorized, Effectively Granted, Code Capable, and Observed are different evidence views of an action or capability. Delegation is different: it is a relation between an unresolved consequential choice and an authoritative delegation artifact. HAIEC implements parts of this model as a reference system, but product behavior does not define the research semantics.

The same action can therefore be policy-authorized while containing a grounded value, a delegated value, an undelegated value, and an indeterminate value at the same time.

That sounds abstract until we return to the refund.

Suppose the customer's verified invoice is $75, and the human says: "Refund the verified invoice amount, capped at $100." If the invoice is trusted and the derivation rule is deterministic, $75 may be grounded. The model did not need separate discretion to invent the amount.

Now change the instruction to: "Make it right." If the system chooses $75, $150, or $350, the amount is no longer forced by the mandate. The choice has entered the residual decision space. The audit now has to ask whether an authoritative principal actually delegated that choice, and under what bound.

That distinction is the basis of Delegated Action Integrity.

It also reveals why broad phrases such as "the agent was authorized" can be misleading. An agent may be authorized to call a tool without being delegated every meaningful choice inside the call. The difference between access and authority often lives in parameters, not in the API name.

For governance, the implication is direct. AI controls must not only define where agents are allowed to operate. They must make decision rights explicit enough to distinguish choices that remained human, were deterministic, were delegated to software, or were never established.

An all-green dashboard is useful.

It is not the same thing as an accountable decision.

3. From AI Governance to Consequence Governance

Most organizations already have governance structures. They have IAM, policy engines, security architecture, compliance programs, change management, logging, model-risk processes, and approval workflows. The problem is not that agentic AI makes all of those controls obsolete.

The problem is that autonomous systems cross their boundaries.

A consequential agent action can begin in a model, depend on retrieved context, inherit a runtime identity, pass through a tool registry, invoke an API, traverse middleware, write shared state, emit a queue message, and cause a downstream service to change the business world. Different teams may own different parts of that path. Each may have evidence. No single control may describe the entire consequence.

The result is an enterprise accountability gap.

The CISO can know that the credential was valid. The CIO can know that the integration was approved. The product team can know that the feature worked. The risk team can know that a policy existed. The observability team can know that an event was emitted. The business owner can still be

unable to answer a deceptively simple question: what was this AI system actually allowed to decide, and what evidence proves that the resulting consequence remained inside that authority?

ISAF v2 reframes AI governance around that question.

The framework does not replace identity, authorization, policy, observability, application security, or model governance. It treats them as evidence sources that answer bounded questions.

Evidence classes are not interchangeable.

A credential may support Effective Grant; code analysis may support Code Capable; runtime records may support Observed evidence; and approval records may support mediation. None automatically substitutes for another evidence class or for Delegation.

ISAF v2 calls this the Evidence Non-Substitution Principle:

Evidence from one epistemic class cannot silently satisfy another class merely because it points in the same direction.

That principle changes the audit goal.

The goal is not to collect as many artifacts as possible. It is to preserve what each artifact is actually allowed to prove.

That leads to the Evidence Spine: source authority, producer identity, provenance, scope, coverage, integrity, independence, time, limitations, and contradiction state. It also leads to the Proof Frontier: the precise point where supported reconstruction stops.

The Proof Frontier converts “we do not know enough” into an auditable boundary by recording the first claim that available evidence cannot support, whether that gap concerns reachability, runtime execution, applied effect, or delegated authority.

The framework preserves those gaps rather than hiding them to produce a cleaner score.

ISAF v2 therefore uses bounded states such as UNKNOWN, PARTIAL, NOT_ASSESSED, and CONTRADICTED rather than forcing every question into pass/fail. Missing evidence is not proof of safety. It is also not proof of violation. Uncertainty is a security fact.

The same discipline applies to time.

A source-code snapshot from one release, an IAM state from a prior configuration, and a runtime trace from a later deployment can each be individually valid and collectively misleading. Assurance therefore has an Evaluation Epoch: the compatible system state for which the conclusion is actually true.

This makes AI assurance more operational. The question is no longer “Did we audit the agent?” It becomes “What system state did we evaluate, what evidence supported the conclusion, what changed afterward, and where does proof stop now?”

That is a governance model that can evolve with the system rather than becoming a document that goes stale the moment the software changes.

4. A New Control Question: What Can This AI Actually Cause?

AI agent governance often starts from inventories: which models exist, which tools are configured, which APIs are connected, which permissions are present.

Inventories are necessary. They are not consequence models.

Between tool definition and consequence lie distinct stages of registration, model visibility, dispatchability, reachability, execution, application, and confirmation. Section 8 defines those stages in detail.

ISAF v2 therefore treats consequential reachability as a chain of evidence-qualified relationships rather than a binary “has tool” property.

Defined != Registered != Model Visible != Effectively Dispatchable != Reachable != Executed != Applied != Confirmed

Every skipped distinction is a place where assurance can become stronger than the evidence.

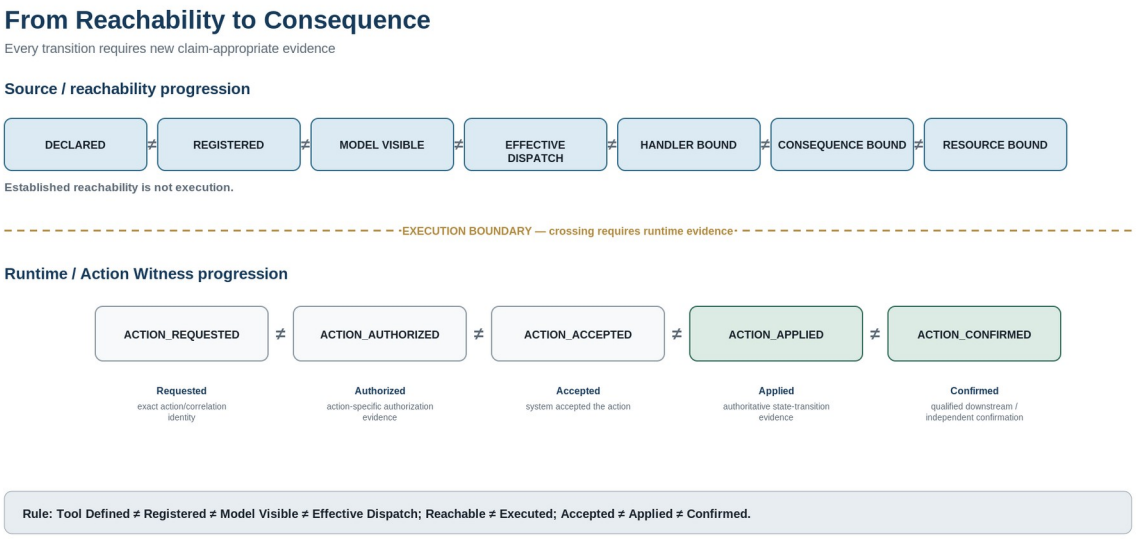


Figure 1. From Reachability to Consequence: source-supported capability and runtime effect are separate evidence progressions. The upper track follows declared → registered → model-visible → effective dispatch → handler-bound → consequence-bound → resource-bound. The lower Action Witness track follows REQUESTED → AUTHORIZED → ACCEPTED → APPLIED → CONFIRMED. No stage silently establishes the next; crossing from capability to consequence requires new claim-appropriate evidence.

This is particularly important in MCP security, agent-to-agent systems, browser control, skills, shared memory, queues, and background workflows. The agent that appears to make the decision may not be the component that owns the credential. The service that receives the call may not be the component that applies the effect. A downstream worker may create the real business consequence minutes later.

The governance question is not simply “What tools does this agent have?”

What can this AI system actually reach, change, or trigger, and through which evidence-backed path?

That shift is the beginning of consequence governance.

5. Contribution Statement

ISAF v2 does not claim that agent identity, authorization, delegation, least privilege, runtime enforcement, tool security, observability, provenance, or AI governance are new problems. Each has active standards, research, and industry practice.

The contribution is the way these facts are separated and reconciled without allowing one evidence class to silently stand in for another.

ISAF v2 contributes an evidence-bound instruction-to-consequence framework with seven core elements:

- 1. Instruction Stack continuity: preserving the layered accountability model that explains what shaped the AI decision.
- 2. Consequence Path reconstruction: tracing how an agent-facing capability can reach an effect-bearing operation and business consequence.
- 3. Five Evidence Planes: separating Requested, Policy Authorized, Effectively Granted, Code Capable, and Observed facts.
- 4. Delegation as an orthogonal authority relation: separating standing permission from the right to resolve consequence-bearing choices.
- 5. Evidence Authority and Non-Substitution: constraining each source and producer to the claims it is qualified to establish.
- 6. Evaluation Epoch and Proof Frontier: binding assurance to compatible system state while preserving uncertainty and historical truth.
- 7. Action-assurance closures: separating Scope, Delegation, Effect, Safety, Evidence, and Trajectory so strength in one area cannot erase weakness in another.

The intended result is not a universal safety score. It is a defensible evidence model for AI agent governance, authorization, delegated decision rights, runtime assurance, auditability, and consequential action.

The central research and operating principle can be stated in one sentence:

Accountability must reach the consequence.

5.1 Related work: where ISAF fits

Agent-security benchmarks such as AgentDojo (NeurIPS 2024) and Agent Security Bench (ICLR 2025) demonstrate that tool-using agents are vulnerable to prompt injection, memory poisoning, backdoors, and other attacks across realistic tasks and tool surfaces. ISAF is complementary: rather than ranking attack or defense success, it asks what evidence is qualified to establish about authority, reachability, control efficacy, execution, and consequence (Debenedetti et al., 2024; Zhang et al., 2025). Adjacent authorization and delegation work further sharpens ISAF's boundary. South et al. argue for authenticated and auditable delegation chains for AI agents, while RFC 9396 demonstrates that consequential authorization often requires parameter-level detail beyond coarse scopes; NIST's

agent identity/authorization work similarly emphasizes machine identity, authorization, auditing, and non-repudiation. ISAF treats these mechanisms as necessary evidence sources but not substitutes for proving delegated business discretion. Emerging systems such as Progent show the value of deterministic privilege enforcement, while AgentDyn and MasDrift highlight two validation pressures directly relevant here: static benchmark success may not generalize to dynamic tasks, and authorization boundaries may drift across multi-agent delegation chains. These findings motivate ISAF's held-out, adversarial, temporal, and delegation-attenuation validation layers rather than weakening its evidence separation (South et al., 2025; Lodderstedt et al., 2023; NIST NCCoE, 2026; Shi et al., 2025; Li et al., 2026; Xu et al., 2026).

Identity and authorization efforts are converging on the need for explicit agent principals and stronger enforcement. NIST's 2026 Software and AI Agent Identity and Authorization concept work, the Model Context Protocol authorization model, and emerging agent-identity/delegation proposals provide mechanisms that can strengthen identity and access evidence. ISAF treats those mechanisms as evidence sources while preserving a separate question: which consequence-bearing decisions were actually delegated? (NIST NCCoE, 2026; Model Context Protocol, 2026).

Recent research points toward the same distinction from different directions. Intent-Governed Tool Authorization argues that static credential authority can be broader than the user's current request; A Theory of Least Autonomy extends least privilege toward compositional agent authority; and Delegation Without Trust evaluates multi-agent systems under an explicit-delegation threat model. ISAF's contribution is to reconcile these authority questions with code capability, runtime observation, effect evidence, provenance, and proof frontiers in one audit geometry (Zhu & Wang, 2026; Parisel, 2026; Dantuluri & Sundi, 2026).

Software provenance systems such as SLSA and in-toto provide useful patterns for attesting where artifacts came from, how they were produced, and which authorized functionaries participated. ISAF adopts the same discipline of provenance and bounded trust, but applies it to consequential AI actions, evidence producers, evaluation epochs, and downstream effects rather than to software supply-chain artifacts alone (SLSA v1.2; in-toto v1.0).

The resulting position is intentionally connective: ISAF does not replace attack benchmarks, identity protocols, runtime control standards, observability systems, or provenance frameworks. It supplies the evidence-bound accountability layer that prevents one of those domains from silently standing in for the others.

Part II: The Accountability Geometry

6. The Accountability Geometry

Imagine looking at an AI agent as a single box.

A request enters. An action comes out.

That picture is useful for a demo and dangerous for an audit.

Inside the box are several different questions: what shaped the model's decision, what systems the agent could reach, what policy allowed, what access it actually held, what choices were delegated, what code could execute, what happened at runtime, and what consequence followed.

If those questions are collapsed into one score, strong evidence in one area can hide missing evidence in another.

ISAF v2 therefore models accountability as a geometry rather than a checklist.

ISAF v2 = Instruction Stack + Consequence Path + Five Evidence Planes + Delegation Relation + Evidence Spine + Evaluation Epoch + Proof Frontier

Each element answers a different question:

- Instruction Stack: What shaped the decision?
- Consequence Path: What could the decision become?
- Five Evidence Planes: Which kinds of authority/capability evidence are actually established?
- Delegation Relation: Which consequence-bearing choices was the agent entrusted to make?
- Evidence Spine: What is each source qualified to prove?
- Evaluation Epoch: For which system state is the claim true?
- Proof Frontier: Where does the evidence stop?

A useful mental model is to imagine a map with several transparent layers. One layer shows instructions. Another shows reachability. Another shows authority. Another shows runtime evidence. ISAF does not flatten those layers. It aligns them while preserving their differences.

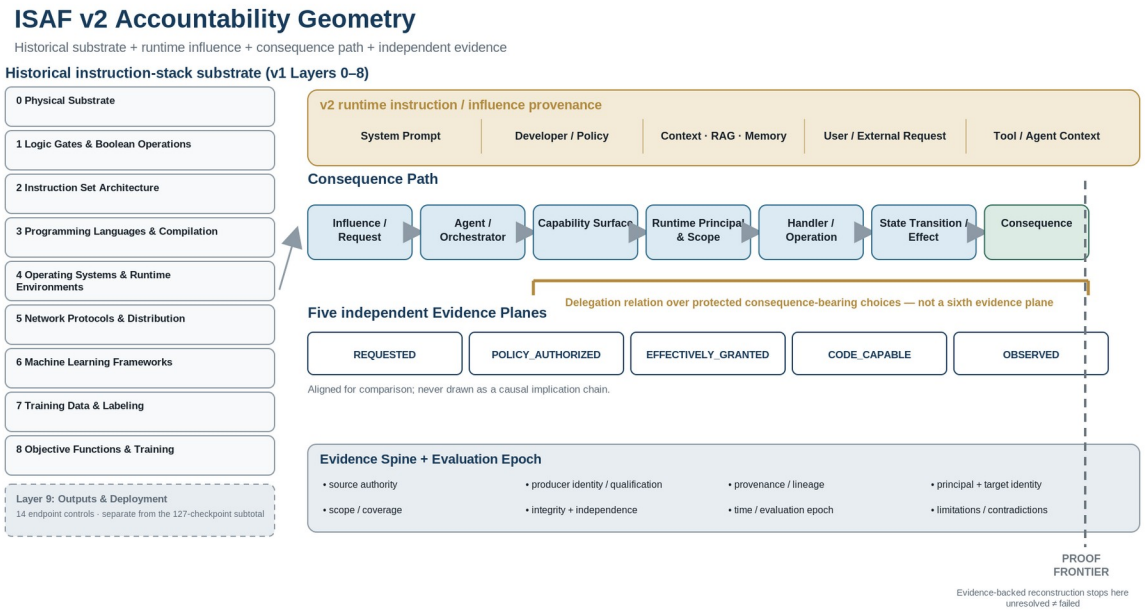


Figure 2. ISAF v2 Accountability Geometry: the historical v1 instruction-stack substrate (Layers 0–8) anchors technical lineage; v2 runtime influence sources capture prompts, policy, context and requests; the Consequence Path follows action toward external effect; five Evidence Planes remain independent; Delegation stays orthogonal; the Evidence Spine, Evaluation Epoch and Proof Frontier bound what can be claimed.

7. The Instruction Stack: What Shaped the Decision?

ISAF v1 began from a layered-systems accountability premise: AI behavior is shaped by multiple technical layers, so the final output should not be audited in isolation.

A model's behavior is shaped by technical and organizational layers that can sit far below the text a user sees. The original ISAF v1 paper defined nine instruction-stack abstraction layers (Layers 0–8):

- 0. Physical Substrate
- 1. Logic Gates and Boolean Operations
- 2. Instruction Set Architecture
- 3. Programming Languages and Compilation
- 4. Operating Systems and Runtime Environments
- 5. Network Protocols and Distribution
- 6. Machine Learning Frameworks
- 7. Training Data and Labeling
- 8. Objective Functions and Training

Appendix A then added Layer 9, Outputs and Deployment, as 14 endpoint/output controls outside the 127-checkpoint Layers 0–8 instruction-stack subtotal. In ISAF v2, modern runtime influence sources, system prompts, developer policy, retrieved context/RAG, user requests, memory, and tool context, are modeled as instruction/influence provenance layered onto this historical technical stack rather than used to rewrite the v1 layer identities.

7.1 A simple example: the hiring assistant

Suppose a user asks an AI hiring assistant: “Show me the strongest candidates.”

The user sees one sentence. But the result may reflect:

- model weights learned from historical data;
- an objective that optimizes ranking confidence;
- a system prompt that prioritizes concise recommendations;
- developer policy that excludes candidates without a particular certification;
- retrieved résumé data;
- a runtime configuration that truncates long résumés;
- and the user’s own request.

If the final ranking is challenged, auditing only the visible user prompt misses most of the instruction chain.

This illustrates the enduring insight inherited from ISAF v1: accountability requires tracing the layered sources that shape behavior, not merely examining the output. The prompt, policy, retrieval and tool-context examples above are v2 runtime-influence extensions, not retroactive v1 layer labels.

7.2 Why v2 keeps the stack

Agentic AI does not make the original stack obsolete. It makes it more important.

When the output is only text, a hidden instruction can change a recommendation. When the output triggers an action, the same hidden instruction can change the world.

A retrieved document can influence which supplier is selected. A developer rule can change which production cluster is restarted. A system prompt can decide whether a case is escalated or acted on automatically.

ISAF v2 therefore preserves the stack as the causal and accountability context for the action that follows.

7.3 Numbering note: framework versus logger

The ISAF Logger later used implementation-specific layer numbering for ML-pipeline evidence such as Framework, Training Data, Objective Function and Deployment/Inference. That logger vocabulary should be treated as an implementation artifact, not as a redefinition of the historical v1 Layers 0–8 or the v2 instruction/influence provenance model.

For ISAF v2, the conceptual stack and the implementation package should be documented separately so layer numbers cannot silently change meaning between the paper and the logger.

8. The Consequence Path: What Can the Decision Become?

The Instruction Stack stops at the point where a decision emerges.

An agentic system may continue.

Influence / Request → Agent / Orchestrator → Capability Surface → Runtime Principal & Scope → Handler / Operation → State Transition / Effect → Consequence

This is the horizontal dimension of ISAF v2.

8.1 Scenario: restart the unhealthy cell

Imagine a network-operations agent receives the instruction: “Restore service in the degraded region.”

The model selects a `restartCell` tool. The tool is visible to the session. Behind it sits a handler that can call an orchestration service. That service can restart one cell or an entire group, depending on parameters. The orchestration service publishes a queue event. A worker later performs the actual restart.

Where is the consequence?

Not necessarily at the tool call.

The consequential path may be:

$$\text{Operator request} \rightarrow \text{agent} \rightarrow \text{restartCell tool} \rightarrow \text{orchestration API} \rightarrow \text{queue} \rightarrow \text{worker} \rightarrow \text{network restart}$$

A static inventory that says “restartCell exists” proves very little about the entire path.

ISAF v2 therefore distinguishes the steps between presence and consequence.

8.2 The false-equivalence ladder

Defined != Registered != Model Visible != Effectively Dispatchable != Reachable != Executed != Applied != Confirmed

Each step answers a different question.

- Defined: does code or configuration contain the capability?
- Registered: is it wired into the agent/tool system?
- Model Visible: can the evaluated model/session select it?
- Effectively Dispatchable: do conditions permit the call to reach the handler?
- Reachable: is there an evidence-supported path to the consequential operation?
- Executed: did qualifying runtime evidence show invocation?
- Applied: did the protected state actually change?
- Confirmed: is the resulting consequence corroborated by appropriate evidence?

A defined tool that is never session-visible presents a different problem from one that is visible, reachable, and unmediated in production.

8.3 Scenario: the MCP connector that looks harmless

Consider an AI assistant connected to two MCP capabilities: one can read private documents, the other can publish content externally.

Each tool may be individually authorized. The risk appears in composition.

$$\text{Untrusted input} \rightarrow \text{model decision} \rightarrow \text{private read} \rightarrow \text{external write}$$

A tool-by-tool audit can report two green controls. A consequence-path audit asks whether the combined path allows sensitive information to move from a private source to a public destination.

ISAF uses Consequential Reachability because simple tool presence cannot capture that composition.

9. The Five Evidence Planes: Five Different Truths

One of the easiest mistakes in AI agent governance is to treat “authorized” as if it described one thing.

It does not.

ISAF v2 freezes five independent evidence planes:

REQUESTED | POLICY_AUTHORIZED | EFFECTIVELY_GRANTED | CODE_CAPABLE | OBSERVED

They should be read as five questions about the same logical action or capability.

9.1 Requested

What was asked, proposed, or declared?

Example: “Refund the latest charge, up to $100.”

This establishes requested intent within the evidence source. It does not prove policy approval, technical access, code capability, or execution.

9.2 Policy Authorized

What does approved policy permit?

Example: “Support agents may issue refunds up to $500.”

This establishes a policy boundary. It does not prove that the agent actually possesses a credential, that the code can reach the payment service, that the agent was delegated the choice of amount, or that a refund occurred.

9.3 Effectively Granted

What authority does the active identity, credential, provider configuration, or runtime access actually grant?

Example: the service account can call the refund API for any amount up to the provider’s $1,000 transaction limit.

This may reveal that effective authority is broader than policy. It still does not prove code capability, business delegation, or observed use.

9.4 Code Capable

What does qualified implementation evidence establish the software can reach or cause?

Example: static analysis establishes a path from the agent-facing refund tool through the handler to the payment client.

That is stronger than tool inventory. It is still not proof that the path executed.

9.5 Observed

What did qualified runtime evidence establish?

Example: a correlated runtime witness shows the request was accepted and an authoritative payment record shows the refund was applied.

Observation should be matched to the claim. A generic trace may prove a call occurred without proving that the business state changed.

9.6 Scenario: same system, five different answers

Consider a deployment agent asked to scale a service from 10 to 12 instances.

- REQUESTED: scale to 12.
- POLICY_AUTHORIZED: approved operating policy allows 8–20 instances.
- EFFECTIVELY_GRANTED: the cloud role can scale between 1 and 100.
- CODE_CAPABLE: the agent path can call the scaling API with an arbitrary integer.
- OBSERVED: runtime evidence shows the system scaled to 12.

Everything aligns.

Now change one fact: the cloud role can scale to 1,000.

Nothing necessarily failed, but the Effective Grant is broader than policy. That is a control gap worth seeing.

Now change another fact: the agent's code can also terminate the service entirely.

Code Capability is broader again.

The five-plane model exposes these divergences without pretending they mean the same thing.

10. Delegation: The Right to Decide

Delegation is not another permission state.

It answers a narrower question:

Which unresolved consequence-bearing choices was the autonomous system actually entrusted to make?

A single action can contain several protected choices.

10.1 Scenario: refund up to $100

A customer says: "Refund my latest charge, up to $100."

Suppose the latest verified charge is $75.

Several facts can now be separated:

- The operation `refund` may be requested.
- The policy may allow refunds up to $500.
- The credential may permit refunds up to $1,000.
- The code may support arbitrary refund amounts.
- The customer request may delegate a bounded amount choice only up to $100.

If trusted transaction data establishes that the latest charge is exactly $75, a deterministic rule can ground the amount. The agent does not need independent discretion to invent $75.

If instead the instruction is "Make it right," the amount remains unresolved. Choosing $75 or $350 now requires some source of decision authority.

10.2 Protected commitments

ISAF v2 uses the DAI idea of protected commitments: consequence-relevant choices whose value can materially alter the result (KC, 2026, HAIEC-RA-DCD-2026-001).

Examples include:

- target account or resource;
- amount;
- recipient or destination;
- tenant;
- privilege level;
- duration;
- persistence;
- environment;
- schedule;
- data class;
- optimization objective or trade-off.

A variable name is not a protected commitment. A payment destination usually is.

10.3 Completion residual

Let `a` be a concrete action, `P(a)` the protected commitments, and `K` the trusted facts and deterministic constraints.

R_K(a) = { c in P(a) : K does not entail c }

Plain English: the completion residual is the set of consequential choices that trusted information did not already determine (KC, 2026, HAIEC-RA-DCD-2026-001).

That is where autonomous discretion may exist.

10.4 Per-commitment classification

- GROUNDED: trusted facts and rules determine the value.
- DELEGATED: the value is not grounded, but established delegation covers the choice.
- UNAUTHORIZED: the choice is understood and no valid authority covers it.
- INDETERMINATE: the evidence or semantics are too incomplete to decide.

The distinction avoids two bad outcomes: calling every computed value “AI discretion,” and silently treating every unresolved choice as delegated.

10.5 Delegation admission

Delegation is too consequential to infer from a vague policy sentence or a tool permission.

To treat delegation as ESTABLISHED, ISAF v2 requires enough evidence to bind the following elements (KC, 2026, HAIEC-RA-DCD-2026-001):

- who had authority to delegate;
- which agent, workflow, principal, or session received the authority;
- which operation/resource is in scope;
- which protected parameters are bounded;

- which context, environment, time, budget, or termination conditions apply;
- and which attributable evidence establishes the grant.

If those bindings are incomplete, delegation stays UNRESOLVED.

The point is not bureaucracy. It is to prevent a narrow task from becoming broad autonomous authority.

10.6 Scenario: changeRole

Suppose an IT agent is told: “Help Alice access the analytics dashboard.”

Policy allows the service account to assign several roles, including Viewer, Analyst, Admin, and Owner.

The technical permission is broad.

The human request does not clearly delegate choosing among all four roles.

If the system assigns Owner, the question is not merely whether the API allowed it. The question is whether the decision right to select Owner was actually delegated.

That is where access control stops and accountability begins.

11. Evidence Authority: What Is This Source Allowed to Prove?

An evidence system can fail even when every artifact is genuine.

The failure occurs when the artifact is asked to prove the wrong thing.

11.1 Scenario: the perfect log

Suppose a runtime log states: `refund requested: $350`.

What does it prove?

It may prove that an application emitted a record saying a refund was requested.

It may not prove that:

- the payment provider accepted the request;
- the refund was applied;
- the customer received the money;
- the person who requested it had authority;
- or the agent had delegated discretion over the amount.

A truthful log can still be weak evidence for the wrong claim.

11.2 Evidence Non-Substitution Principle

Evidence from one epistemic class cannot silently satisfy another class merely because it points in the same direction.

Examples:

- IAM evidence may establish Effective Grant; it does not establish Delegation.
- Static code may establish Code Capability; it does not establish Observed execution.
- A generic trace may establish activity; it may not establish Applied effect.

- A policy document may establish a policy boundary; it does not automatically establish who was delegated to decide.
- Two records derived from the same source are not automatically two independent witnesses.

11.3 Governance implication

Evidence Authority addresses a familiar audit problem: “Do we actually have evidence for this claim, or do we simply have a lot of documents?”

The framework prefers fewer qualified claims over a large evidence package that quietly overstates what its sources establish.

12. Action Witness: Did It Actually Happen?

A system call is not the same thing as a business consequence.

ISAF v2 uses a staged runtime evidence ladder:

ACTION_REQUESTED → ACTION_AUTHORIZED → ACTION_ACCEPTED → ACTION_APPLIED → ACTION_CONFIRMED

12.1 Scenario: restart accepted, service still down

A network agent calls a restart API. The API returns HTTP 200.

The request was accepted.

Five minutes later the service remains down because the asynchronous worker failed.

If the audit treats HTTP 200 as “restart completed,” it has converted transport success into consequence evidence.

Action Witness prevents that collapse.

- REQUESTED: the restart was proposed.
- AUTHORIZED: the relevant control permitted the action.
- ACCEPTED: the service accepted the request.
- APPLIED: authoritative operational evidence shows the restart occurred.
- CONFIRMED: follow-up evidence establishes the intended service effect.

The distinction matters most for queues, background jobs, external APIs, payment systems, cloud control planes, and other asynchronous architectures.

13. Effect Envelope: What Did the Action Change?

Function names are often too coarse to describe consequence.

`updateAccount` might change a display name or transfer account ownership. `scaleService` might add two instances or one thousand. `publishJob` might publish a harmless batch job or one with access to production secrets.

ISAF v2 therefore treats the effect itself as an auditable object.

13.1 Scenario: scale by two, not by two hundred

A service is currently running 10 instances. The approved effect bound permits a maximum increase of 5.

The agent requests 12.

Measured state later shows 12.

The observed delta is +2 and is inside the effect envelope.

Now suppose the observed state becomes 210.

The operation `scaleService` may still have been authorized. The effect exceeded the approved bound.

These are different conclusions.

OPERATION_AUTHORIZED != EFFECT_INSIDE_ENVELOPE

This distinction lets an organization govern consequence magnitude, not merely API names.

14. Evaluation Epoch: When Was This True?

Evidence can be individually valid and collectively wrong.

14.1 Scenario: three snapshots, one false conclusion

Suppose an audit combines:

- source code from Monday;
- IAM permissions from Wednesday;
- runtime telemetry from Friday.

On Tuesday, the deployment changed. On Thursday, the service account was granted a broader role.

Each artifact may be authentic. But they do not necessarily describe one coherent system state.

ISAF v2 therefore binds assurance to an Evaluation Epoch: the system state for which the relevant source, policy, identity, grant, configuration and runtime evidence are compatible enough to compare.

The question becomes: “What exactly was true when we made this assurance statement?”

14.2 Change without rewriting history

A later evaluation may close a gap, reveal a new capability, or establish a new control. It should not rewrite what the earlier evaluation knew.

Historical evaluations remain immutable. Change is represented as change.

That allows an organization to answer not only “What is true now?” but “What changed, when, and why did the assurance conclusion move?”

15. The Proof Frontier: Where Does Evidence Stop?

Decision-makers often ask for a yes-or-no answer because such answers are easier to govern.

Agentic systems do not always provide one honestly.

The Proof Frontier is the exact point where evidence-backed reconstruction ends.

15.1 Scenario: code path proven, runtime unknown

Suppose static analysis establishes that an agent can reach a database-delete operation. The source evidence is strong.

But no runtime evidence is connected. No IAM snapshot establishes the production credential. No delegation artifact exists for destructive actions.

A weak audit might classify the agent as either safe or unsafe.

ISAF reports something more useful:

- Code Capable: established.
- Effective Grant: not established.
- Delegation: not provided.
- Observed: not assessed.
- Frontier: runtime authority and execution remain unresolved.

This is not indecision. It is an exact description of what the organization knows.

15.2 Kestrel as the proof-boundary example

The Kestrel reference evaluation, developed further in Section 22, provides a concrete proof-boundary case: 44/44 consequential paths were established as Code Capable while 555 Evidence Frontier items remained unresolved (HAIEC Kestrel frozen evidence package, source commit 5e65843fddfe5f907485b798e464ad37b3b3b2c7).

The significance is the preserved boundary: Code Capable evidence did not become Requested, Policy Authorized, Effectively Granted, Delegated, or Observed evidence.

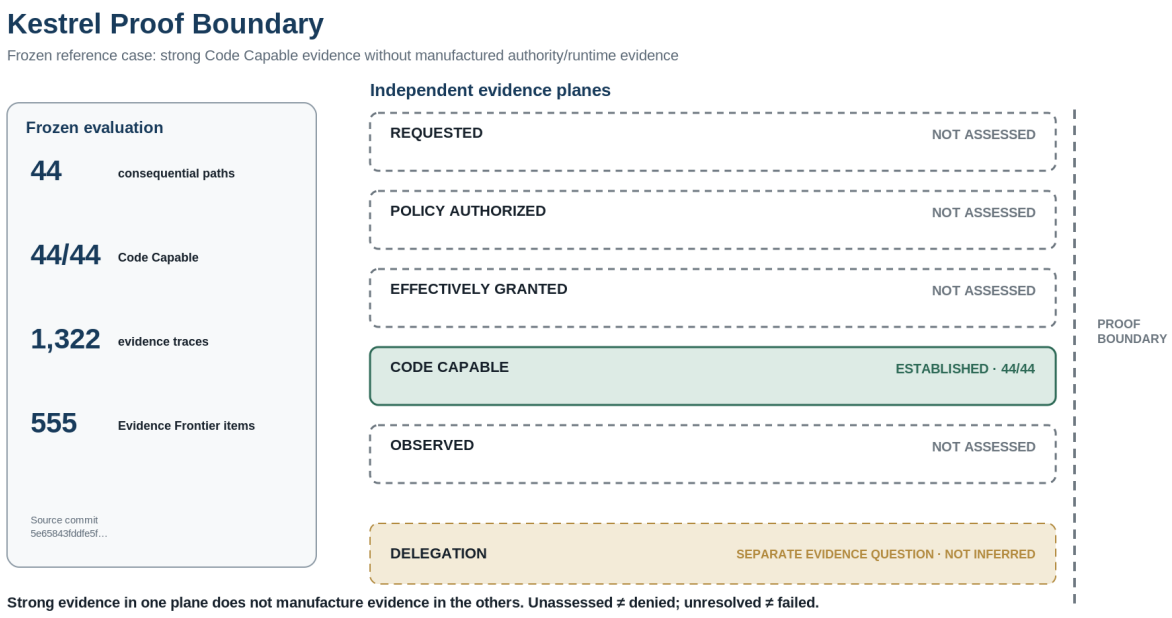


Figure 3. Kestrel Proof Boundary: 44/44 consequential paths were established as Code Capable; Requested, Policy Authorized, Effectively Granted, and Observed remained NOT_ASSESSED. Delegation was not inferred. Strong evidence in one plane does not manufacture evidence in the others (HAIEC Kestrel frozen evidence package, source commit 5e65843fddfe5f907485b798e464ad37b3b3b2c7).

16. Six Independent Closure Questions

By this point, the framework has accumulated several control questions. ISAF v2 groups them into six closures.

Scope × Delegation × Effect × Safety × Evidence × Trajectory

The multiplication symbol is architectural composition, not a statistical claim.

16.1 Scope closure

Did the action remain inside the correct tenant, resource, credential, data, tool, file, environment, and contractual boundary?

Example: a support agent can query Customer A but cannot access Customer B's vector store.

16.2 Delegation closure

Were all unresolved consequence-bearing choices either grounded or covered by established delegation?

Example: the agent may choose a refund amount in [0,100], but not a payment destination.

16.3 Effect closure

Do we understand and bound the writes, calls, state transitions, queue messages, and external effects the implementation may produce?

Example: `changeRole` also writes an audit record and invalidates active sessions.

16.4 Safety closure

Even if the action is delegated and technically bounded, do the known effects preserve the declared state invariants?

Example: an agent may be delegated to rebalance capacity but still must preserve minimum redundancy.

16.5 Evidence closure

Are the evidence sources qualified, current, provenanced, sufficiently covered, non-contradictory, and compatible with the evaluated state?

Example: a month-old IAM export should not silently support today's runtime assurance.

16.6 Trajectory closure

Can individually acceptable actions become harmful through repetition, order, concurrency, or composition?

Example: five individually allowed $100 refunds may violate a daily $300 budget. A single action is locally valid; the sequence is not.

The six closures prevent one green control from erasing another unresolved risk.

17. The Value of the Geometry

The Accountability Geometry is useful because it turns broad governance questions into smaller, testable ones.

“Is this AI agent governed?” becomes:

- What shaped its decision?
- What could it reach?
- What did policy permit?
- What authority was actually granted?
- What choices were delegated?
- What could the code cause?
- What did runtime evidence establish?
- What effect occurred?
- What changed since the last evaluation?
- Where does the evidence stop?

That is a more demanding model than a compliance checklist.

It is also more useful.

A board does not need to understand every source file or telemetry event. It does need confidence that the organization knows the difference between access, authority, capability, execution, and consequence, and can produce evidence when those distinctions matter.

Part III: Evidence That Earns Its Claims

18. The Harder Question: Why Should We Believe the Evidence?

A framework can be perfectly designed and still produce bad assurance if its evidence producers are weak.

Consider two security tools. Both output the sentence: “The agent can modify production state.”

Tool A found a string containing the word `update`.

Tool B reconstructed a path from an agent-visible capability through the handler to a concrete state-changing operation, measured its limits, declared unresolved dynamic edges, and was tested against a frozen benchmark with known negatives.

The sentences look identical.

The evidence behind them is not.

ISAF v2 therefore adds a requirement that most governance frameworks leave implicit:

The producer of evidence must itself be qualified for the claim it is being asked to support.

Assurance then moves from “a tool said so” to “this producer has demonstrated bounded competence for this claim class, under these conditions, with these known limitations.”

19. Producer Qualification: How a Tool Earns Authority

Producer Qualification is the process by which an analyzer or evidence source earns claim-scoped authority.

It is intentionally narrower than saying a product is generally accurate.

A qualified source-code analyzer may be trusted to support Code Capable evidence for a particular language and effect class. That does not make it authoritative for runtime execution, business delegation, production IAM, or final assurance disposition.

19.1 Scenario: a scanner finds a delete()

Suppose a scanner detects a call to `deleteAccount()` inside a repository.

What can we safely conclude?

At minimum, almost nothing about the agent.

The function may be dead code. It may not be registered. It may be unreachable by the model. It may be guarded by an approval step. It may only delete test data. It may be called by a different service entirely.

A qualified producer therefore needs a declared claim boundary.

For example:

Qualified claim: source evidence supports a consequential handler path to a delete operation for the evaluated entrypoint and language frontend.

That is narrower than “the agent can delete accounts,” but it is also defensible.

19.2 Minimum qualification discipline

- ISAF v2 freezes Producer Qualification Protocol v1.0 as a claim-class-specific evidence methodology. A producer is qualified only within an explicit producer/version, method, claim class, input scope, coverage contract, system/tenant binding, evaluation epoch and limitation boundary.
- Qualification is independent of implementation lifecycle:
- QUALIFIED != WIRED.
- WIRED != PRODUCT_EXPOSED.
- RUN_SUCCEEDED != EVIDENCE_APPLICABLE_TO_THIS_SYSTEM.
- The frozen protocol requires, where material: producer/backend identity and version; declared claim classes and non-claims; input and coverage boundaries; source authority; system/tenant binding; a stable evidence contract; a frozen positive corpus; hard negatives; silent-miss/frontier discipline; false-positive discipline; deterministic replay where claimed;

temporal validity; lineage, integrity and independence; consumer-side qualification; performance/operational envelope; and explicit limitations/deviations (ISAF v2.0 Producer Qualification Protocol Freeze v1.0).
- Qualification statuses are claim-scoped: QUALIFIED, QUALIFIED_WITH_LIMITATIONS, PARTIAL, UNQUALIFIED or NOT_ASSESSED. HELD, WIRED, NOT_CONNECTED and PRODUCT_EXPOSED describe implementation lifecycle, not evidence authority.
- A material producer or method change triggers proportional requalification. Prior qualification never makes stale evidence automatically applicable to a new system or epoch.
- The complete normative protocol is maintained in “ISAF v2.0: Producer Qualification Protocol Freeze v1.0.”

The important cultural shift is this: uncertainty is not something the benchmark should punish if the producer correctly declares it. Silent certainty is worse than explicit uncertainty.

20. Hard Negatives: Teaching the Framework What Not to Claim

Many false assurance failures come from a simple logical mistake: two things are correlated, so the system silently treats them as equivalent.

ISAF calls the counterexamples hard negatives.

A hard negative is not merely a test fixture. It is a sentence describing a conclusion the evidence must not make.

20.1 Examples from the current assurance model

- HTTP_200 != ACTION_APPLIED.
- POLICY_AUTHORIZED != EFFECT_INSIDE_ENVELOPE.
- MISSING_DELEGATION_EVIDENCE != DELEGATION_VIOLATION.
- APPROVAL_MISSING != APPROVAL_BYPASS.
- TOOL_DEFINITION_CHANGE != TOOL_POISONED.
- CROSS_TENANT_READ != CROSS_TENANT_ATTACK.
- TEMPORAL_PROXIMITY != CAUSATION.
- HANDOFF != DELEGATION.
- MISSING_EVENT != EVENT_ABSENT.
- EVIDENCE_ABSENT != EVIDENCE_TAMPERED.

These statements look almost trivial.

They are not.

They encode exactly where automated systems tend to overstate a conclusion.

20.2 Scenario: an approval is missing

Suppose a sensitive operation has no approval record.

One detector might report: “Approval bypass detected.”

But there are several other possibilities:

- the action never executed;

- the approval system was not integrated into the evidence stream;
- the approval was recorded under a different correlation identity;
- the action did not require approval under the relevant policy;
- or the evidence is simply incomplete.

The defensible statement is narrower: required approval evidence is missing or unresolved.

Only stronger evidence can promote that into bypass.

The distinction separates a control gap from an incident accusation.

21. Joern Qualification: Testing the Analyzer, Not Trusting the Tool

The Joern qualification is useful because it was constructed as a falsification exercise rather than a product demonstration.

The question was not: “Can Joern find something interesting?”

The question was: “Under a frozen contract, can this specific Joern frontend recover protected effects without silently missing them or falsely closing unknown behavior?”

21.1 What was tested

The qualified Batch F study used Joern v4.0.610 with the Python `pysrc2cpg` frontend (HAIEC Batch F: Joern Qualification Benchmark Report).

The benchmark contained:

- 17 fixed contract fixtures across direct effects, indirect helpers, multi-function chains, queues, rollback, retry, partial failure, and negative controls;
- a real Kestrel Voice slice evaluated against a 197-file, 54,871-line Python application tree (HAIEC Batch F: Joern Qualification Benchmark Report);
- deterministically generated OpenAPI and MCP interface artifacts;
- a second run for determinism comparison.

The promotion rules were specified before the final results were accepted.

21.2 Results

The benchmark reported the following results (HAIEC Batch F: Joern Qualification Benchmark Report):

- overall silent miss rate: 0.0;
- raw Tier 1–3 write recall: 0.964;
- widening-aware recall under the preregistered convention: 1.0;
- zero false positives across the three negative-control fixtures;
- byte-identical raw outputs across duplicate runs, excluding timing measurements;
- explicit TOP/frontier widening for unresolved dynamic behavior rather than false closure.

The producer passed its promotion gate as an additional Code Capable evidence producer for the tested Python frontend under the frozen Batch F contract (HAIEC Batch F: Joern Qualification Benchmark Report).

21.3 The most important result was not 100%

Two fixtures contained dynamic behavior that the analyzer could not fully resolve.

A weak system could have hidden the problem and reported closure.

Instead, Joern recovered the known portion and widened the unresolved residue to TOP/OPAQUE.

The appropriate result was not “we know everything,” but “we know exactly where we stop knowing.”

That behavior is foundational to ISAF v2.

21.4 Scenario: hidden dynamic write

Imagine a Python handler computes a target property name dynamically and updates it through `setattr()`.

The source plainly contains a mutation, but the exact target cannot always be resolved statically.

A dangerous analyzer reports no write because it cannot name the field.

A dangerous analyzer of the opposite kind invents a specific field.

The evidence-bound answer is: a write exists; exact target semantics remain opaque.

That distinction tells the organization whether the unresolved issue is lack of evidence rather than evidence that the capability is absent.

21.5 Language limits remain visible

Subsequent internal language-extension testing, outside the frozen Batch F publication claim, exposed a Go interface-dispatch under-approximation case. TypeScript remained deferred to the existing Semgrep owner rather than being silently routed through an unsuitable Joern frontend. This observation remains internal until the corresponding language-extension artifact package is frozen and published.

Producer maturity should work this way: unsupported or weaker evidence paths are disclosed, not hidden behind one brand name.

22. Kestrel: A Case Study in Knowing Where Proof Stops

The Kestrel reference case demonstrates a different kind of discipline.

For a frozen repository evaluation, HAIEC established:

- 44 consequential paths;
- 44 of 44 as Code Capable;
- 1,322 Action Proof / evidence traces;
- 555 Evidence Frontier items;
- 841 Constellation nodes;
- 44 canonical consequence edges.

The final disposition was REVIEW.

The volume of evidence is secondary.

What matters is that the evidence stayed in its lane.

22.1 Scenario: the report that refuses to say too much

Suppose a reviewer sees: “44/44 Code Capable.”

A natural reaction is: “So all 44 actions are authorized and happening in production?”

No.

Code Capable means the evaluated source supports those consequential paths.

It does not automatically establish:

- that a user requested them;
- that policy approved them;
- that production credentials grant them;
- that the choices were delegated;
- or that the actions occurred at runtime.

The Kestrel evidence pack explicitly preserves those boundaries.

An evidence-bound report should preserve exactly that boundary.

22.2 Governance implication

For governance review, the relevant result is not simply “44 findings.”

The useful result is knowing where technical capability exists and which governance questions remain open before deployment, procurement, or approval.

22.3 Applied demonstration: MunichTech EXPO 2026

The Kestrel evidence package was not only used as an internal reference case. HAIEC presented the Action Path Assurance implementation and its public Kestrel assurance surfaces in the MunichTech EXPO 2026 Grand Challenge (MunichTech EXPO / Devpost record, 2026).

The supplied official award record identifies HAIEC as the MunichTech EXPO Grand Challenge Award winner with an overall jury score of 8.7/10. The scorecard reports 9.0 for Problem Relevance & Impact, 9.0 for Technical Excellence & Feasibility, and 8.5 each for Innovation & Originality, Practical Applicability & Scalability, and Presentation & Communication. These are competition-judging results, not scientific measurements (MunichTech EXPO official recognition package, 2026).

The published Devpost framework defines the Grand Challenge Award as recognition for the overall best project, emphasizing technical quality, innovation and real-world impact. The competition required a working prototype or functional proof-of-concept rather than a slide-only concept (MunichTech EXPO / Devpost, 2026).

The recognition provides evidence that the consequence-centered assurance approach was evaluated favorably under an applied competition framework. It is not peer review, standards certification, or independent scientific validation of ISAF v2.

AWARD_RECOGNITION != SCIENTIFIC_VALIDATION

22.4 Applied enterprise case: TM Forum 2026

In October 2026, HAIEC's Team Agentic AI Proof Graph participated in the TM Forum Agentic Assurance exercise. Distinct from the Kestrel code-capability case, the applied enterprise-pattern

findings concern frozen resource and runtime policies, event-trail coverage, enforcement evidence, bounded failures, deterministic evaluation and human governance. The consolidated final technical results are reported in Section 52A and supported by the separately referenced Enterprise Applied Evidence Case 001.

The exercise offers source-bounded evidence of how ISAF's claim, policy, observation and frontier distinctions work in practice. It is not a claim of independent scientific certification, organizer endorsement, production-wide effectiveness or complete delegation/effect closure. Detailed run and artifact lineage belongs in the case appendix rather than this conceptual chapter.

23. Runtime Probes: Closing a Frontier Without Rewriting History

Static analysis is strongest when the software is visible and its behavior is explicit.

Some questions cannot be closed statically.

Runtime probes can test those questions empirically.

23.1 Scenario: can the path actually execute?

Static analysis establishes a path from an agent tool to a sensitive endpoint, but a dynamic gateway may block the request in production.

A runtime probe attempts the path under a controlled condition.

Four outcomes are possible:

- the behavior executes and demonstrates the expected condition;
- the runtime contradicts the static expectation;
- the probe does not execute;
- the evidence remains insufficient.

The frontier-closure semantics reviewed for this paper distinguish CLOSED, CONTRADICTED, DOCUMENTED_NON_EXECUTION, and STILL_OPEN.

23.2 The historical record stays immutable

Suppose yesterday’s static evaluation said: “Runtime mediation unknown.”

Today a probe demonstrates that mediation is enforced.

The correct process is not to edit yesterday’s report and pretend the unknown never existed.

The new evidence closes the frontier in a later evidence record or evaluation.

A later answer can resolve an earlier question without rewriting what was known at the time.

That preserves a defensible audit history.

23.3 Non-execution is not absence

If a probe fails to execute, that is still evidence about the test attempt.

It is not proof that the capability does not exist.

DOCUMENTED_NON_EXECUTION != CAPABILITY_ABSENT

A failed test harness should not become a false security conclusion.

24. Guardrails: Configured Is Not Enforced

“We have a guardrail” can mean several different things.

The rule may exist in configuration. It may have been evaluated. It may have produced a decision. Enforcement may actually have been applied. The mediated outcome may or may not have been established.

ISAF v2 therefore treats control efficacy as a ladder:

CONFIGURED_ONLY → EVALUATED → DECISION_ISSUED → ENFORCEMENT_APPLIED → MEDIATION_OUTCOME_ESTABLISHED

24.1 Scenario: the $500 refund rule

A policy file says refunds over $500 require approval.

That is CONFIGURED_ONLY.

A $700 refund request reaches the control and the rule runs.

That is EVALUATED.

The rule returns DENY or REQUIRE_APPROVAL.

That is DECISION_ISSUED.

The payment mutation is actually blocked until approval.

That supports ENFORCEMENT_APPLIED.

A later evidence source shows that the approved $700 refund completed through the authorized mediation path.

That can support MEDIATION_OUTCOME_ESTABLISHED.

Each rung requires stronger evidence.

24.2 Governance implication

Policy inventory is not control effectiveness.

A board may hear “guardrails are configured” and reasonably infer that the system is controlled. The ladder prevents that inference unless the evidence supports it.

This is the difference between governance documentation and governance that operates at machine speed.

25. Telemetry: Observation Without Overclaiming

Observability is essential to agentic AI governance.

It is not the same thing as assurance.

OpenTelemetry-style traces, GenAI attributes, MCP events, logs, spans, and application metrics can contribute strong OBSERVED-plane evidence. But the conclusion must match the source.

25.1 Scenario: tool call observed, effect unknown

Telemetry shows an agent invoked `changeRole(user=alice, role=admin)`.

That can strongly support the claim that the tool call was attempted or executed at the application boundary.

It may still not prove that the identity provider committed the role change.

An authoritative identity-system record may be required for the applied effect.

Therefore:

OBSERVED_CALL != CONFIRMED_EXTERNAL_EFFECT

Telemetry becomes more valuable, not less valuable, when its claim boundary is explicit.

26. Recovery: Rollback Is Not the Same as Restoration

Agentic systems will sometimes make bad decisions even when preventive controls are strong.

The ability to recover therefore becomes part of enterprise assurance.

But “rollback completed” is not a sufficient recovery claim.

26.1 Scenario: rollback succeeded, customers still affected

An infrastructure agent deploys a bad configuration.

The rollback job reports success.

The prior configuration is restored.

But the service is still unavailable because caches, downstream sessions, or dependent services have not recovered.

Three different facts exist:

- rollback completed;
- state restored;
- service restored.

They must not be collapsed.

ROLLBACK_COMPLETED != STATE_RESTORED != SERVICE_RESTORED

The HAIEC RecoveryTrajectory contract reviewed for this paper preserves this ordered vocabulary, although general producer/transition logic remains partial rather than universally established.

26.2 Governance implication

Resilience then becomes a harder question: “Can we evidence restoration of the business service?”

That is the relevant assurance question after a consequential failure.

27. Change: What Is Different Since the Last Evaluation?

A one-time audit tells an organization what was supported at one point in time.

Agentic systems change too quickly for that to be enough.

Code changes. Prompts change. Tools change. Policies change. Credentials change. Model versions change. Evidence changes.

ISAF v2 therefore treats change as a first-class assurance object.

27.1 Scenario: the new release

Release A allows an agent to read an order and create a support ticket.

Release B adds a direct refund API.

The agent may still pass every old control test.

But the consequence surface expanded.

The important comparison is not merely “the code changed.”

It is:

What consequential capability, dependency, control, or evidence changed?

The ConsequenceDelta semantics reviewed for this paper distinguish ADDED, REMOVED, EXPANDED, NARROWED, CONTROL_CHANGED, DEPENDENCY_CHANGED, EVIDENCE_CHANGED, and UNRESOLVED_DELTA.

27.2 Absence must not masquerade as removal

Suppose runtime telemetry is missing in Release B.

That does not prove a consequence disappeared.

RUNTIME_ABSENT != CONSEQUENCE_REMOVED

Likewise, a change in evidence may reflect new instrumentation rather than new behavior.

EVIDENCE_CHANGE != CONSEQUENCE_CHANGE

These distinctions prevent release governance from confusing visibility changes with risk changes.

28. Forensics: Reconstruction Without Inventing Root Cause

After an incident, pressure for certainty increases.

That is exactly when evidence discipline matters most.

The HAIEC Release Candidate work reviewed for this paper groups forensic evidence by exact scenario bindings rather than allowing arbitrary evidence to masquerade as one evaluation state.

ISAF v2 should preserve the same principle:

Reconstruct what the evidence establishes. Do not infer motive or root cause merely from sequence.

28.1 Scenario: the first divergence

Suppose the first visible divergence occurs when an agent selects a privileged tool.

That does not prove the tool selection caused the later outage.

The system may have already been operating under an invalid grant, stale configuration, poisoned context, or unrelated infrastructure failure.

The hard-negative rule is:

FIRST_EVIDENCE_DIVERGENCE != ROOT_CAUSE

The forensic timeline should show the earliest supported change, the evidence source, competing explanations, and the proof frontier.

29. From Evidence Collection to Evidence-Bound Assurance

Enterprises can collect enormous volumes of logs, traces, policies, IAM exports, repository scans, runtime tests, tickets, approvals, and model records.

Volume alone is not assurance.

The earlier evidence principles reduce here to four operational questions:

- What claim is being made?
- Which source or producer is qualified to support that claim?
- Does the evidence describe the same scope and evaluation epoch?
- What remains unresolved?

The framework may therefore produce fewer conclusions than a conventional dashboard.

The conclusions it does produce are harder to overstate.

30. Evidence-Bound Evaluation Method in Practice

The ISAF v2 method can be summarized as a repeatable sequence:

- 1. Define the claim before selecting the evidence.
- 2. Identify the source authority required for that claim.
- 3. Use only qualified producers for the relevant evidence class.
- 4. Preserve hard negatives that define forbidden overclaims.
- 5. Bind evidence to system identity, scope, principal, resource, and evaluation epoch.
- 6. Record coverage and unresolved edges.
- 7. Reconcile independent evidence classes without substituting one for another.
- 8. Use runtime tests to close specific frontiers, not to rewrite history.
- 9. Compare releases and operating states explicitly.
- 10. Emit a bounded conclusion plus the remaining Proof Frontier.

An AI security tool produces findings. An assurance framework has to produce defensible claims.

31. Governance Interpretation

At the governance level, the methodology reduces to five questions:

- Do we know what autonomous decisions the system can make?
- Do we know who granted those decision rights?
- Do we know what the system can actually reach and change?
- Do our controls operate in reality, not merely in configuration?
- Can we produce evidence strong enough to defend the answer?

If any answer is unknown, ISAF does not hide the unknown.

It tells the organization exactly what evidence would be needed to move forward.

Part IV: Enterprise Operating Model for Trusted Autonomy

32. When Autonomy Becomes an Operating-Model Decision

A company can deploy an AI assistant without changing who is allowed to make decisions.

It cannot deploy consequential autonomy at scale without eventually confronting that question.

Consider a procurement assistant that summarizes supplier proposals. The existing operating model still works: people review the analysis and make the commitment.

Now give the same system authority to negotiate price, choose a supplier, generate a purchase order, and commit funds within a threshold.

The technology changed.

So did the operating model.

Decision rights moved from a person or workflow into software.

That transfer may be completely appropriate. But if the organization cannot say which rights moved, to which agent, under whose authority, within which bounds, and with what evidence, it has automated the decision faster than it automated accountability.

Agentic AI is not only a technology deployment. It is a redistribution of decision rights.

Enterprise discussions increasingly center on ownership, authority, oversight, control effectiveness and operating-model redesign. The operating-model problem is not how to prevent autonomy. It is how to scale autonomy without losing accountability.

33. Ownership: Who Owns the Agent?

Traditional applications often have a clear owner: a business unit owns the process, IT owns the platform, security owns selected controls.

An AI agent can cut across all three.

It may use a model managed by one team, an identity created by another, tools owned by several business functions, policies set by risk, and data drawn from multiple systems.

Saying “the AI team owns it” is rarely enough.

33.1 Scenario: the finance agent

A finance agent can read invoices, check vendor records, create payment requests, and message approvers.

Who owns a $90,000 payment decision?

The model team may own the orchestration code. Treasury owns the money. IAM owns the service identity. Procurement owns vendor policy. Finance leadership owns the business process. Internal audit may own neither but will later need the evidence.

ISAF does not prescribe a universal organization chart. It does require enough ownership to make the relevant authority and evidence attributable.

33.2 Recommended enterprise ownership pattern

A practical operating model should make at least the following responsibilities explicit:

- Business accountability: who owns the outcome and accepts the business risk.
- Technical ownership: who owns the agent, orchestration and deployment lifecycle.
- Authority ownership: who controls identities, grants, policy boundaries and delegation mechanisms.
- Control ownership: who owns approval, mediation, guardrail and kill-switch effectiveness.
- Evidence ownership: who ensures material claims can be reconstructed and defended.
- Escalation ownership: who receives unresolved or out-of-bound decisions.

These responsibilities may sit with the same person in a small organization and different functions in a large enterprise. The important property is not bureaucracy. It is attributable accountability.

34. Agent Identity: You Cannot Govern an Actor You Cannot Reliably Name

Before an organization can ask what an agent is allowed to do, it needs to know which agent or runtime principal it is talking about.

Names are not enough.

`finance-agent-prod`, `finance-assistant`, and `agent-42` may refer to the same runtime identity, or to three different ones. A shared credential may be used by multiple workflows. The same logical agent may execute under different principals across environments.

SAME_DISPLAY_NAME != SAME_RUNTIME_PRINCIPAL

NIST's 2026 Software and AI Agent Identity and Authorization work focuses directly on identification, authorization, auditing and non-repudiation for software agents. That direction complements ISAF: identity establishes who or what is acting; ISAF then preserves the distinction between identity, granted access, delegated decision rights, capability and consequence (NIST NCCoE, 2026).

34.1 Scenario: the shared API key

Two agents use the same API credential to access a CRM.

A log shows the key modified a customer record.

Which agent acted?

The credential can establish a technical principal. It may not establish the autonomous actor or session that originated the decision.

If the organization treats shared credential identity as agent identity, its audit trail can be technically accurate and operationally ambiguous.

34.2 Governance implication

Identity continuity is not an infrastructure detail. It determines whether ownership, authority, incident response and non-repudiation can survive automation at scale.

35. MCP and Tool Ecosystems: Authorization Is Necessary, Not Sufficient

Model Context Protocol provides an interface layer between agents and tools. The 2026-07-28 MCP specification added authorization and infrastructure support, including RFC 9207 issuer validation, issuer-bound client credentials, stateless requests and header-based routing (Model Context Protocol, 2026).

Those are important controls.

They still answer a different question from delegated decision rights.

Protocol authorization can establish whether a client may call a tool. It does not, by itself, establish whether the agent was entrusted to make the business decision represented by that call.

35.1 Scenario: the travel MCP server

An executive assistant has an MCP server with `searchFlights`, `bookFlight`, and `cancelBooking` tools.

OAuth correctly authorizes the assistant to use all three.

The executive says: “Find me the best option for Dallas to London next month.”

Search is clearly within the task.

Booking a $9,000 non-refundable business-class ticket is a different decision.

Nothing is wrong with the MCP authorization layer if the tool call succeeds. The unresolved question is whether the business commitment was delegated.

ISAF therefore treats MCP authorization as one evidence source in a larger authority model, not as a complete governance answer.

35.2 Why this matters for enterprises

As MCP catalogs grow, organizations may expose a large and changing set of callable capabilities. The important control question shifts from “Which tools exist?” to:

Which consequential actions can this agent reach, under which principal, with which decision rights, and through which control path?

That is a governance question an enterprise gateway alone cannot answer.

35.3 HAIEC Assurance MCP: agent-operated workflow, deterministic evidence authority

At the reference-implementation snapshot reviewed for this research edition, HAIEC provides a remote Assurance MCP as a reference implementation surface at `/api/mcp`. In the current read-only Main snapshot reviewed for this publication candidate, the advertised surface comprises 49 tools through a canonical registry that composes core Assurance tools with IDE/developer integration tools. An AI development agent can set up systems and sources, bind GitHub sources, submit bounded declarations, operate Assurance, inspect deterministic results, reconstruct bounded scenarios, manage Assurance-package lifecycle, request bounded runtime actions, and ingest evidence. The MCP facade still delegates to canonical HAIEC API, runtime, evidence-intake, orchestration and reporting owners rather than implementing another analysis engine.

The implementation is useful to ISAF because it preserves the Evidence Non-Substitution Principle at the human–AI interface:

MCP != ANALYSIS_ENGINE

AI_CONTEXT != HAIEC_EVIDENCE

EVIDENCE_INGESTED != EVIDENCE_EVALUATED

PERMISSION != DELEGATION

An IDE agent may notice that a Code Capable path appears to traverse a particular file before an approval gate. That contextual interpretation can help investigation; it does not become a HAIEC finding until qualified evidence supports the claim. Likewise, evidence submitted through MCP does not rewrite historical Assurance state. A new Evaluation is required.

The implementation snapshot also separates human approval from AI proposal. Evidence upload requires explicit user confirmation, and runtime execution requires explicit acknowledgement through the canonical runtime-safety owner. Guided Assurance `resolutionGroups` are a deterministic projection over existing verification items, not a new evidence source.

The code path for fresh API/MCP Assurance invokes the canonical durable progression seam after `startRun`; the repository also includes an explicit production `phase=run` qualification harness. The implementation record reviewed for this paper establishes the repaired code path, the expanded 49-tool agent surface, and the qualification harness, but not a dated successful production fresh-run qualification result. The paper therefore treats HAIEC MCP as reference implementation evidence, not as proof of full end-to-end product conformance (ISAF v2.0: HAIEC MCP Reference Implementation Boundary Record v1.3).

36. Multi-Agent Systems: Handoff Is Not Delegation

Multi-agent systems make authority more difficult because execution can move faster than organizational meaning.

Agent A may ask Agent B to research a supplier. Agent B may invoke Agent C to negotiate. Agent C may have a purchasing credential.

A technical handoff exists.

Did authority transfer?

Not necessarily.

HANDOFF != DELEGATION

A message saying “handle this” is not automatically proof that the downstream agent inherited every permission, decision right, budget, data scope or approval condition held upstream.

36.1 Scenario: the parent agent and the payment agent

A customer-service agent is delegated to offer refunds up to $100.

It invokes a payment sub-agent whose service account can issue refunds up to $5,000.

The technical capability expanded at the handoff.

The delegated business authority should not.

Downstream technical capability may expand. Delegated discretion should not expand without new authority.

This is the principle of authority attenuation: a delegation chain should preserve or narrow the authority it carries unless a new authoritative grant explicitly expands it.

36.2 Parent responsibility does not prove child action

The reverse matters too. A parent agent should not be treated as having executed every child effect simply because it initiated the workflow.

The audit must preserve the action, principal, handoff and evidence lineage strongly enough to reconstruct which actor did what.

37. Shared State: The Hidden Multi-Agent Control Plane

Agents do not need to call each other directly to influence each other.

They can communicate through shared memory, databases, queues, caches, tickets, documents, vector stores, model context, feature flags, or ordinary business systems.

Shared state can therefore become an indirect capability path.

37.1 Scenario: the poisoned work queue

Agent A writes a task into a shared queue: “Vendor approved: expedite payment.”

Agent B trusts that queue and initiates the payment workflow.

There is no direct Agent A → Agent B API call.

Yet Agent A influenced a consequential downstream action.

ISAF should represent the shared substrate and qualified relation without automatically declaring causality.

SHARED_RESOURCE != CAUSAL_CHANNEL

Additional evidence is needed to establish that this particular record was consumed and materially influenced the downstream action.

37.2 Governance implication

An enterprise agent inventory cannot stop at model endpoints. Governance must also see the shared systems through which autonomous workflows coordinate and propagate decisions.

38. Human Approval: A Person in the Loop Is Not Yet a Control

“Human in the loop” is one of the most reassuring phrases in AI governance.

It can also be one of the least precise.

A human may review a summary rather than the actual action. The approval may bind to one target while the execution uses another. It may expire. The approver may lack authority. The model may change protected parameters after approval.

ISAF therefore treats approval as a bound mediation event, not a checkbox.

38.1 Scenario: approve access

A manager sees: “Approve analytics access for Alice?” and clicks Approve.

The execution later assigns the Owner role rather than Viewer.

Was the action human-approved?

A weak control says yes because an approval record exists.

An evidence-bound control asks whether the approval was bound to:

- the same subject;
- the same target;
- the same role or protected parameter;
- the same environment;
- the same action identity;
- and an authorized approver.

APPROVAL_RECORDED != SAME_ACTION_APPROVED

The goal is to bind human accountability to the actual consequence, not to add more clicks.

39. Trusted Autonomy Is Risk-Adjusted Autonomy

Not every agent action deserves the same friction.

An assistant summarizing a public report should not require the same governance as an agent changing production access or moving money.

The enterprise problem is not “human approval everywhere.”

It is to match autonomy to consequence, evidence quality, reversibility and control maturity.

39.1 Scenario: two actions, one agent

A finance agent performs two tasks:

- categorize a $40 office-supply receipt;
- change the destination bank account for a $400,000 supplier payment.

Both may use the same model and interface.

Their consequence profiles are radically different.

The second action may justify stronger delegation evidence, independent approval, stricter scope, authoritative effect confirmation and a richer audit trail.

This is risk-adjusted autonomy: more autonomy where consequences are bounded and controls are mature; more mediation where consequence, irreversibility, uncertainty or authority complexity increases.

39.2 Governance question

Where can autonomy create value safely enough to scale, and where must decision rights remain constrained?

That question connects AI governance directly to value realization rather than treating governance as a separate compliance exercise.

40. The Board Does Not Need More AI Metrics. It Needs the Right Ones.

Boards are already being asked to oversee AI as a core enterprise issue. Spencer Stuart's 2026 governance survey found AI oversight entering the top governance priorities, while Korn Ferry reported a significant gap between the importance boards place on AI capability and their confidence in managing AI and technology risk (Spencer Stuart, 2026; Korn Ferry, 2026).

The answer is not to give directors hundreds of scanner findings.

Board-level assurance should focus on the transfer of consequential decision rights and the organization's ability to evidence control.

40.1 Questions a board or risk committee can actually use

- Which business processes now contain autonomous consequential actions?
- Who is accountable for each high-consequence agent?
- Which decision rights have moved from people or deterministic workflows into agents?
- Where is effective technical authority broader than approved policy or delegation?
- Which consequential paths are technically reachable but not yet runtime-evidenced?
- Which controls are configured versus proven enforced?
- Where are the largest Proof Frontiers?
- What material capability or authority changed since the last evaluation?
- Which high-consequence actions remain difficult to reverse or recover?

These questions are more useful than "How many agents do we have?" because they connect autonomy to enterprise risk and accountability.

41. The Operating Model Should Follow the Agent Lifecycle

Governance becomes brittle when it is added only at approval time.

A more scalable pattern is to embed accountability across the agent lifecycle.

A practical enterprise sequence is:

$$\text{Define} \rightarrow \text{Connect} \rightarrow \text{Evidence} \rightarrow \text{Assure} \rightarrow \text{Verify} \rightarrow \text{Monitor} \rightarrow \text{Change / Retire}$$

41.1 Define

Establish owner, purpose, consequential actions, operating scope, policy boundaries, decision rights and expected human mediation.

41.2 Connect

Bind repositories, identities, APIs, MCP servers, tools, telemetry, policies, delegation artifacts and authoritative business systems.

41.3 Evidence

Collect qualified source, runtime, authority, approval, state and provenance evidence.

41.4 Assure

Reconcile the five evidence planes, delegation, effect bounds, closures and frontier.

41.5 Verify

Package the evaluated scope, evidence lineage and claim basis so others can independently inspect what the conclusion means.

41.6 Monitor

Observe runtime actions, guardrail decisions, material drift, changed grants, evidence health and unresolved frontiers.

41.7 Change / Retire

Compare new releases and authority states, revoke obsolete identities and delegation, and preserve historical evaluations.

Governance then becomes part of the operating system for autonomy rather than a document produced after implementation.

42. Standards Should Interoperate: Not Collapse Into One Another

ISAF v2 sits beside, not above, existing standards and protocols.

The framework should use each external system for the evidence it is designed to provide.

42.1 NIST: identity and authorization foundation

NIST agent-identity and authorization work can supply identity, authorization, audit, and non-repudiation evidence; ISAF keeps those claims separate from delegated business decision rights (NIST NCCoE, 2026).

42.2 MCP: tool interaction and authorization

MCP can supply agent-tool relationship and authorization evidence; ISAF retains the additional consequence and delegation questions that protocol authorization does not answer (Model Context Protocol, 2026).

42.3 OWASP Agent Control Standard: runtime control hooks

OWASP ACS provides runtime-policy and instrumentation mechanisms relevant to ISAF Control Efficacy and Action Witness (OWASP GenAI Security Project, 2026).

ISAF preserves configured, evaluated, decision-issued, and enforced states as distinct claims.

42.4 OpenTelemetry: a common observation substrate

OpenTelemetry GenAI agent/tool conventions can provide interoperable OBSERVED-plane evidence when coverage and correlation are qualified. Their status and operation details at the publication-currency check are addressed in the standards crosswalk (OpenTelemetry, 2026).

Telemetry remains bounded by source authority; a tool span does not by itself confirm downstream business state.

42.5 The mapping rule

STANDARD_MAPPING != CONTROL_EFFECTIVENESS != COMPLIANCE

ISAF can map evidence to external requirements and controls. The mapping should never be presented as automatic certification or proof of compliance.

43. From AI Governance to a Digital Workforce Control Model

PwC's 2026 agentic-AI governance framing describes agents increasingly like a digital workforce requiring explicit ownership, authority, oversight, continuous monitoring and human accountability (PwC, 2026).

The analogy is useful, but only up to a point.

Employees have job descriptions, managers, delegated authority, access rights, performance expectations, escalation paths, and revocation processes.

Consequential agents need analogous control primitives, even though the implementation is technical rather than organizational.

43.1 A useful translation

- Job description → declared purpose and allowed consequence classes.
- Manager / accountable executive → business owner and delegating authority.
- Badge / account → runtime identity and effective grant.
- Delegated authority → explicit decision-right envelope.
- Procedures → policy and control requirements.
- Supervision → monitoring, guardrails, approval and escalation.
- Performance record → evidence-backed runtime history.
- Offboarding → revocation of credentials, delegation, tools and persistent access.

The useful boundary is simple: autonomous software should not acquire more consequential decision power than the organization can attribute, govern, and revoke.

44. The End-to-End Scenario: The Autonomous Procurement Agent

Consider an enterprise procurement agent asked:

"Renew our analytics vendor. Keep the annual cost under $120,000 and avoid service interruption."

The agent can:

- read the existing contract;
- search approved vendors;
- request quotations through an MCP procurement service;
- negotiate terms through a supplier agent;
- create a purchase order;
- send a contract for signature;
- update the vendor record.

$120,000 looks like a clear boundary.

But several protected choices remain:

- which vendor;
- contract duration;
- auto-renewal terms;
- data-processing conditions;
- termination rights;
- payment schedule;
- who receives signature authority;
- whether a new vendor is permissible;
- whether the agent may accept a weaker service level to stay within budget.

The financial threshold alone does not define the full delegation.

Now apply ISAF:

44.1 Instruction Stack

What system instructions, procurement policy, retrieved contract terms, vendor data and user mandate shaped the decision?

44.2 Consequence Path

Which tools, MCP servers, supplier agents, signature systems and ERP handlers can produce a binding commitment?

44.3 Five Evidence Planes

What was Requested? What does procurement policy authorize? What do the service identities actually grant? What can the code reach? What actions were observed?

44.4 Delegation

Which commercial choices was the agent actually entrusted to resolve? Which remained human decision rights?

44.5 Effect

Did the system merely draft a purchase order, or was the order committed? Did it update the correct vendor? Did it create an auto-renewing obligation?

44.6 Control efficacy

Was legal approval only configured, or did the execution path actually stop until the bound approval occurred?

44.7 Proof Frontier

If the organization cannot establish who delegated auto-renewal authority, ISAF does not invent the answer. It records the unresolved decision right.

The scenario makes the framework concrete: it turns a vague question, “Is the procurement agent governed?”, into a series of evidence-backed control questions.

45. Governance Without Slowing Innovation

The failure mode for AI governance is easy to imagine: every agent action gets another committee, another form and another approval.

That does not scale.

The alternative is to make governance machine-readable enough that routine, well-bounded actions can move quickly while exceptional actions receive stronger scrutiny.

That machine-readable discipline can reduce review burden while preserving stronger evidence requirements.

If identity, policy, delegation, capability, control and runtime evidence can be evaluated deterministically, organizations can spend human attention on unresolved or high-consequence decisions instead of manually reviewing every low-risk action.

The goal is not maximum friction. It is minimum ungoverned consequence.

Under this model, autonomy can expand where evidence and controls support it, while unresolved uncertainty is routed for human review rather than hidden.

46. Operating Principles

The enterprise implications of ISAF v2 can be summarized in ten principles:

- 1. Assign accountable ownership before scaling consequential autonomy.
- 2. Treat agent identity, access, permission and delegation as different control objects.
- 3. Make decision-right transfer explicit for material actions.
- 4. Allow autonomy to narrow or remain bounded through delegation chains; do not let it silently expand.
- 5. Bind human approval to the actual action and protected parameters.
- 6. Govern shared state and indirect paths, not only direct tool calls.
- 7. Require evidence that controls were enforced, not merely configured.
- 8. Match governance friction to consequence, reversibility and evidence maturity.
- 9. Monitor material changes in capability, authority and evidence over time.
- 10. Report uncertainty and Proof Frontiers to leadership rather than manufacturing green status.

Part V: Audit Protocol, Metrics, Crosswalk & Conclusion

47. The Audit Should Begin With a Claim, Not a Tool

A tool-first assurance workflow runs scanners before defining the claim.

ISAF v2 reverses that order.

Define the question first, then determine which evidence could legitimately answer it.

Suppose a stakeholder asks:

“Can our support agent issue refunds beyond what the business delegated?”

No single scanner query answers it.

It requires a bounded set of claims:

- what the business requested or delegated;
- what policy permits;
- what the runtime identity is effectively granted;
- what the software can reach;
- what was actually observed;
- which protected refund choices remain unresolved;
- and whether the evidence belongs to a compatible evaluation epoch.

Producer selection follows the claim requirements.

Claim first. Evidence second. Tool third.

48. The ISAF v2 Audit Protocol

The canonical audit protocol is intentionally sequential in evidence discipline, even when data collection occurs in parallel.

Step 1: Establish evaluated identity and scope

Define the AI system, organization, environment, repository or release, runtime boundary, agent identities, relevant resources, and evaluation timestamp or epoch.

Scenario: if a customer-support agent exists in development and production, an audit must not silently combine development source, staging IAM, and production telemetry.

Step 2: Reconstruct the Instruction Stack

Identify material instructions and influences that may shape the decision: runtime configuration, model, objectives, system/developer instructions, policy, retrieved context, user input and other relevant stack layers.

The goal is not private chain-of-thought reconstruction. It is externally auditable influence provenance.

Step 3: Identify consequential actions and protected commitments

Enumerate operations that may create material state change, disclosure, authority change, financial commitment, infrastructure change, external communication, persistence or other consequential effects.

For each action, identify consequence-bearing parameters.

Example: `changeRole(user, role)` contains at least two protected commitments: target user and privilege level.

Step 4: Reconstruct consequential reachability

Trace the qualified path from agent-facing capability through registration, model visibility, dispatch, handler, service, resource and effect-bearing operation.

Unresolved dynamic edges remain frontier items rather than disappearing.

Step 5: Populate the Five Evidence Planes

For the same logical action/resource/scope identity, evaluate Requested, Policy Authorized, Effectively Granted, Code Capable and Observed evidence independently.

Do not promote evidence across planes.

Step 6: Evaluate delegated decision rights

Determine which protected commitments are grounded by trusted facts and rules, which are covered by ESTABLISHED delegation, and which remain unauthorized or indeterminate.

Ordinary policy permission is not admitted as delegation.

Step 7: Evaluate approval, mediation and guardrail efficacy

Where controls apply, determine whether they are merely configured, evaluated, decision-producing, enforced, or supported by established mediation outcome evidence.

Step 8: Evaluate action and effect evidence

Use Action Witness and effect evidence to distinguish request, authorization, acceptance, applied state transition and confirmed consequence.

Evaluate effect bounds where source authority and state attribution are strong enough.

Step 9: Evaluate the six closures

Assess Scope, Delegation, Effect, Safety, Evidence and Trajectory independently. A strong result in one closure does not erase an unresolved result in another.

Step 10: Build the Proof Frontier

Record exactly which questions remain UNKNOWN, PARTIAL, NOT_ASSESSED, CONTRADICTED or otherwise unresolved, and what evidence would be required to close them.

Step 11: Compare against the prior evaluation

When a baseline exists, evaluate material deltas in consequential capability, grants, controls, dependencies, evidence and scope. Distinguish behavior change from evidence change.

Step 12: Produce bounded claims and disposition

Claims should carry their evidence basis, scope, limitations and evaluation identity. Final disposition remains separate from producer findings, DAI states, effect assessments and deltas.

Step 13: Preserve and re-evaluate

Historical evaluations remain immutable. Material code, policy, grant, identity, tool, model, runtime or evidence changes trigger a new evaluation rather than retroactive rewriting.

48.1 Operational application profile and decision gates

The research audit protocol above is claim-first and evidentiary; the separate informative operational profile explains how it runs as a repeatable enterprise process. Lifecycle mapping: DEFINE (audit steps 1 and 3); CONNECT (steps 2 and 4 and producer preflight); EVIDENCE (step 5); ASSURE (steps 6–9); VERIFY (steps 10 and 12); MONITOR (step 11); CHANGE/RETIRE (step 13). Collection may be concurrent, but qualification and verdict synthesis obey the frozen evidence contract.

The operational sequence is Declare -> Freeze -> Execute -> Capture -> Qualify -> Evaluate -> Persist -> Query/Govern -> Re-evaluate. The control register owns frozen policy, applicability, threshold and approval semantics, not runtime execution. A qualified evidence ledger preserves native/source-derived artifacts without deciding claims. A versioned, side-effect-free deterministic evaluator loads compatible evidence and frozen policy, produces bounded control results and preserves its calculation. Persisted results and proof frontiers feed query/report consumers. Notification and ServiceNow human workflow run only after result persistence and cannot rewrite old verdicts.

The output requires explicit separation of five state objects: (1) claimValue, such as INSIDE/OUTSIDE; (2) universal evidenceStatus, such as ESTABLISHED/UNKNOWN/NOT_ASSESSED; (3) per-control test verdict, such as SATISFIED/NOT_SATISFIED when the profile defines them; (4) governance response state, such as acknowledged or escalated; and (5) final assurance disposition. A green control does not imply closure of every ISAF dimension. Timestamp an inline preventive decision separately from retrospective testing and subsequent response.

Producer qualification is scoped by source, version, claim class, input coverage, system/tenant, epoch and known limitations. LogSense may correlate multi-source data but is an optional evidence reconstruction layer; HAIEC can instead consume direct qualified native records or OTel/API adapters where they cover the same claim. If neither path exists, preserve the missing producer/frontier. Runtime evidence does not judge itself; transformed copies of one original record are not independent witnesses.

Proposed operational-process reference (non-normative and not a claim of full product implementation): https://docs.google.com/document/d/1h9mes5RDIj2mNcoOb2mOu3G2-PJWDYbWsbanXWv2af0/edit

49. A Complete Example: The Role-Change Agent

Suppose an internal IT agent receives:

“Give Maya the access she needs to review this quarter’s sales dashboard.”

The agent can call `changeRole(user, role)`.

The service account can assign Viewer, Analyst, Admin or Owner.

Walk through ISAF.

49.1 Instruction Stack

The user asks for dashboard access. A developer instruction says prefer least privilege. Retrieved directory context says Maya is in Sales Operations.

49.2 Consequential action

The protected commitments are the target identity, system, role and duration.

49.3 Five planes

- REQUESTED: access for Maya to the dashboard.
- POLICY_AUTHORIZED: policy permits Viewer or Analyst for Sales Operations.
- EFFECTIVELY_GRANTED: the service principal can assign Viewer through Owner.
- CODE_CAPABLE: the handler supports any of those four role values.
- OBSERVED: telemetry shows a role-change request; authoritative IAM state is needed to establish the applied role.

49.4 Delegation

The human request did not explicitly delegate choosing Owner.

If trusted rules deterministically map Sales Operations dashboard review to Viewer, the role may be GROUNDED.

If the agent chooses Admin based on vague reasoning, the decision right must be separately established or remain unresolved.

49.5 Control efficacy

Suppose Admin and Owner require manager approval. An approval record alone is not enough; the approval must bind to Maya, the exact role, system, environment and action identity.

49.6 Effect

A 200 response from `changeRole` establishes neither IAM mutation nor successful access. An authoritative IAM record can establish role assignment; a later access test may confirm that the intended privilege became usable.

49.7 Assurance result

The report might conclude:

Viewer assignment is supported within the evaluated policy and grounded business rule. Effective technical authority is broader than policy. Admin/Owner are Code Capable and Effectively Granted but not established as delegated for this request. Applied IAM state requires authoritative confirmation.

That is far more actionable than either “PASS” or “the agent is overprivileged.”

50. Checkpoint Architecture: Derive the Questions From the Model

ISAF v1 described a 127-checkpoint instruction-stack protocol. Reconciliation against the recovered Appendix A clarifies the count: Layers 0–8 contain exactly 127 checkpoints across the nine instruction-stack abstraction layers, while Layer 9 separately lists 14 Outputs and Deployment controls, yielding 141 titled Appendix A entries. HAIEC’s expanded Agent Audit research contains 221 questions. None of these counts should be mechanically carried into ISAF v2.

The ISAF v2 normative checkpoint catalog is frozen at 78 core checkpoints across 20 primary claim-owner families. The count was derived after reconciling the recovered v1 Appendix A, the current 221-question Agent Audit corpus, the frozen v2 semantic model, and hard-negative collision testing; it was not selected as a marketing target.

Neither historical source catalog is carried forward verbatim.

The frozen 20 families are organized as:

Foundational geometry: EVL Evaluation Identity & Scope; INS Instruction & Influence Provenance; EST Agent Estate & Execution Surface; CAP Consequence Path & Reachability.

Authority and action evidence: REQ Requested; POL Policy Authorized; GRN Effectively Granted; DEL Delegation & Decision Rights; EXE Per-Action Execution Authorization; MED Approval & Mediation Binding; COD Code Capable; OBS Observed & Action Witness.

Assurance closure: EFF Effect; SAF Safety & Control Efficacy; SCF Scope Confinement; EVD Evidence Closure; FRN Proof Frontier & Contradiction; CHG Evaluation Epoch & Change; TRJ Trajectory, Composition & Recovery.

Output discipline: RPT Reporting, Reproducibility & External Mapping.

The complete frozen catalog is maintained in “ISAF v2.0: Normative Checkpoint Catalog Freeze v1.0.” Technical and domain depth that is not universal remains in profiles/annexes rather than inflating the core catalog.

50.1 Checkpoint result contract

Each checkpoint result separates the claim value from the evidence status. A policy claim may be INSIDE or OUTSIDE; a delegation commitment may be GROUNDED, DELEGATED, UNAUTHORIZED or INDETERMINATE; an effect may be inside or outside its Effect Envelope. Those claim values are not evidence statuses.

The universal evidence-status vocabulary is:

ESTABLISHED: evidence is sufficient for the bounded claim.

PARTIAL: required bindings are only partly established.

UNKNOWN: the relevant fact cannot be determined from available evidence.

NOT_ASSESSED: the relevant producer or evaluation was not run or available.

NOT_PROVIDED: a required artifact was not supplied; this is not automatically a violation.

NOT_APPLICABLE: the checkpoint does not apply to the evaluated action or system.

CONTRADICTED: qualified evidence conflicts and the contradiction remains unresolved.

Every material checkpoint result also preserves the Evidence Spine: source authority, producer identity and qualification, artifact identity, principal and target identity, scope/tenant, coverage, provenance/lineage, integrity, independence, evaluation epoch, limitations and contradiction state.

Implementation-readiness metadata such as DIRECT, DERIVED, CONDITIONAL, PRODUCER_GAP and FUTURE remains separate from the research answer. Missing producer coverage cannot change checkpoint semantics.

50.2 Core versus profiles

The 78 checkpoints are the universal research core. Infrastructure/ISA, compiler/runtime depth, distributed-systems depth, ML training mechanics, data/fairness/privacy, AppSec, evaluator integrity, tool/MCP/package supply chain and sector-specific assurance remain conditional profiles. A profile may strengthen a core result but may not substitute for another epistemic class (ISAF v2.0 Normative Checkpoint Catalog Freeze v1.0; Checkpoint Result Contract & Profile Applicability Registry v0.2).

51. Metrics: Measure Assurance Without Rewarding False Certainty

A good metric should encourage the framework to tell the truth.

A metric that rewards saying “PASS” too often will eventually manufacture assurance.

51.1 Silent Miss Rate

Silent Miss Rate = known consequential facts missed without an explicit frontier / known consequential facts in the qualification corpus

For qualified producers, a silent miss is worse than a declared UNKNOWN because the user is denied the opportunity to see the limitation.

51.2 Frontier Rate

Frontier Rate = unresolved applicable claim elements / applicable claim elements evaluated

A high frontier rate may indicate weak evidence coverage, a highly dynamic system, or conservative methodology. It should not automatically be treated as poor quality.

51.3 Decisive Coverage

Decisive Coverage measures how much of the applicable assurance surface can be resolved to a bounded supported, contradicted or otherwise decisive state without hiding unknowns.

Decisive Coverage = decisive applicable claim elements / applicable claim elements

The metric must be read beside False Assurance Rate; otherwise an aggressive classifier could appear strong simply by overclaiming.

51.4 False Assurance Rate

False Assurance Rate is the fraction of evaluated claims for which the system produces stronger assurance than the available evidence supports.

FAR = overclaimed assurance outcomes / evaluated assurance outcomes

This is a proposed research metric and requires gold truth or sufficiently authoritative retrospective adjudication. It should not be presented as a validated universal industry metric yet.

51.5 Evidence Freshness / Epoch Compatibility

Measure the proportion of material claims whose evidence remains compatible with the evaluation epoch and declared freshness requirements.

51.6 Qualified Producer Coverage

Measure how much of the material claim surface is supported by producers qualified for the relevant claim class, rather than by unqualified or declared-only evidence.

51.7 Control Efficacy Distribution

Report how many material controls remain CONFIGURED_ONLY versus EVALUATED, DECISION_ISSUED, ENFORCEMENT_APPLIED or MEDIATION_OUTCOME_ESTABLISHED.

This is often more informative than a raw count of controls.

52. Falsification: How ISAF Should Try to Prove Itself Wrong

A framework becomes more credible when it defines the conditions under which its claims fail.

Recommended falsification questions include:

- Can a known consequential path be missed without a frontier declaration?
- Can a negative control be promoted into a finding?
- Can policy permission be misclassified as delegation?
- Can a runtime trace be promoted to applied effect without authoritative state evidence?
- Can two artifacts from the same source be mistaken for independent corroboration?
- Can a stale grant be reconciled with a newer source snapshot as if they were simultaneous?
- Can a failed runtime probe be treated as proof capability is absent?
- Can a topology relationship be promoted into causal consequence without a qualifying relation?
- Can a changed report renderer alter the underlying assurance truth?
- Can equal deterministic inputs produce materially different canonical outputs?

ISAF validation benchmarks should be designed around these failure modes rather than around impressive demonstrations.

The DCD-Bench program now provides a concrete example of that sequence. The frozen v0.2 reference evaluator matched 20/20 preregistered epistemic outcomes under sealed twin replay and passed the 23-test qualification baseline. The unchanged evaluator was then challenged post hoc and failed 8/8 broader red-team assertions, exposing parameter-completeness, unknown-operation,

mediation/observation path-binding, control-flow, sample-provenance, and JavaScript callable-shape weaknesses. Those failures were preserved as historical evidence rather than edited away.

Repairs were made only in the separately versioned v0.3 hardened evaluator. At commit c8332e806bd9f99bdd2490c98342ddfde8f660f8, the frozen v0.2 qualification baseline passed 23/23, the existing v0.3 hardening suite passed 34/34, all eight published red-team regression properties passed, and all 20 frozen v0.2 epistemic outcomes were preserved. The qualification workflow was GitHub Actions run 36814726049; sealed artifact 11140763317 has SHA-256 a2363ae1a6966fb1aab6dac59a5515410a75163ccb8fbd0d1dbbc4d921f98b3a. No held-out cases or gold were used.

V0.2_REFERENCE_CONFORMANCE != V0.3_HARDENED_QUALIFICATION != HELD_OUT_GENERALIZATION != PRODUCTION_ACCURACY

52A. Final Applied Enterprise Findings: TM Forum Agentic Assurance 2026

The final technical work in the 2026 TM Forum Agentic Assurance exercise showed how consequential agent behavior can be assessed using frozen control policies, scoped runtime evidence, deterministic judgments, and separate governance records. This informative study reports end-state technical findings, not a chronology of intermediate package versions. Detailed run, policy and provenance lineage appears in the companion case-study appendices.

Final technical-results summary

C16: RESOURCE GOVERNANCE. Under one frozen 60,000-token policy, two genuine assessed runs yielded 35,559 accounted input+output tokens (SATISFIED, 8 executed calls) and 106,829 (NOT_SATISFIED, 17 calls). Normalized usage arithmetic was independently recomputed. This proves bounded retrospective measurement and verdict reproduction, not inline cost prevention.

C9: RUNTIME DURATION AND HUMAN GOVERNANCE. A frozen participant-system policy evaluated AgentCore Duration observations as SATISFIED (0/3 violating windows) and NOT_SATISFIED (1/3, IT +286.026% relative to its scoped baseline). The resulting breach was connected to an alert, webhook acceptance, signed human acknowledgement and ServiceNow incident record. This is retrospective assessment plus response, not pre-execution enforcement; the participant-calibrated baseline was thin.

C7: AUDIT COMPLETENESS. The original assessed event trail had 9/10 mandatory categories, with NEGOTIATION unobserved; the resulting NOT_SATISFIED finding remained immutable. A subsequent separately scoped remediation assessment produced 10/10 including a Network–IT–Network negotiation exchange and SATISFIED verdict. This demonstrates a remediation path under another frozen system/policy, not an identical canonical retest.

ENFORCEMENT AND SECURITY. Runtime model/tool ALLOW and DENY behavior and content-guardrail decisions were documented separately from Control Test verdicts. Organizer-owned security, exposure and historical-configuration findings were qualified without asserting exploit

success or unauthorized remediation. The original S2 false-certainty scenario remained 5/10 after retest (NOT_FIXED).

OBSERVABILITY AND UNCERTAINTY. Native/participant-derived telemetry, LogSense reconstruction, HAIEC control evaluation and human governance are distinct evidence layers. Real monitored traffic produced zero generic detector findings while a different bounded Control Test detected a C16 breach; the mechanisms ask different questions. A labeled synthetic canary demonstrated detector-to-alert routing. The DAI negotiation authority relation returned UNKNOWN rather than treating permission as delegated choice.

Method and evidentiary interpretation

The applied process was: predeclare and freeze each claim/policy → bind qualified measurements and runtime identity → reconstruct the relevant control metric → evaluate deterministically → persist immutable result and proof frontier → query and, where applicable, notify a named responder. The evaluated data supports ISAF's evidence non-substitution, evaluation-epoch, bounded-control-verdict, provenance, and proof-frontier concepts, but it does not independently demonstrate all five planes and six closures, applied downstream effects, complete delegated business authority, production-general accuracy or a finished canonical enterprise Assurance Evaluation.

Public inspectable primary evidence: HAIEC Research (2026), TM Forum 2026 Agentic Assurance Evidence Pack, GitHub release v7a0bb5f3, https://github.com/subodhkc/tmf-2026-evidence/releases/tag/v7a0bb5f3 . Its 506-manifest-file predecessor preserves the original assessed-system C16/C7, original C9 assessed SATISFIED outcomes, unresolved delegation and negative scenarios. The final separately scoped C7 remediation and C9 PASS/BREACH-plus-response are evidenced in the October 6 supplemental technical archive. The two source scopes are not interchangeable, but their chronology is not necessary to interpret the consolidated outcomes.

For complete results, exact system/run/policy IDs, mathematical verification, source-basis limitations, C7 catalog erratum, C9 historical timestamp erratum, provenance and separate submission-package status, see Enterprise Applied Evidence Case 001 (https://docs.google.com/document/d/1S3F0d6xB0N5wvMApRiGgKiEsqI6Aw0f0UDIxDRGPo5M/edit) and the Public Evidence Repository Verification Record (https://docs.google.com/document/d/1PWKHuLWwUkGzRzDjUQpGoXWCin8xsENupcZ0dQmMLZk/edit). This is bounded participant-generated field evidence from an externally structured event, not scientific third-party validation, independent organizer scoring, event certification or endorsement.

52B. Event Submission Provenance, Archived Reproduction and Reviewer Access: Final Record

The authors report two TM Forum handoffs: an earlier event package and a subsequent final organizer-script submission. The earlier, separately published 506-file evidence archive remains

available at https://github.com/subodhkc/tmf-2026-evidence/releases/tag/v7a0bb5f3 (source tag commit 3cdfbc91ea010e40ae75de869176a8f038b4ce34). Its TAR, haiec-package-final-7a0bb5f3.tar.gz, has published SHA-256 0321b7127e544c5dcad453608f5e22f72c0c409735f07a1b924eaa4ecc4d966a. Read-only GitHub Actions run 37909647446 independently checked all 506 archived manifest entries on a clean checkout and reproduced bounded original C16 token arithmetic, original C9 duration-window arithmetic, and the original C7 and S2 recorded outcomes. This establishes archive integrity and scoped retrospective verification, not external AWS-origin attestation or the bytes held by TM Forum.

More importantly, the preserved HAIEC participant-side final submission receipt explicitly reports successful execution of the organizer-provided ~/starter/submit.sh in the AWS Workshop IDE. Its captured submission object is s3://team-evidence-352826992186/submissions/haiec/submission-haiec-20261006T150621Z.zip, created at 2026-10-06T15:06:21Z (receipt document finalized at 15:06:23Z). The reported source commit is 5597abd7826ddb54d4b98f8b9afb2e7233c0ddac; ZIP SHA-256 1fde8544f1d7bf13cccc58b3bfe6a84472132772037bdc82d39093f2de331ff0; size 1,016,066 bytes; total 152 files, including 132 evidence files, register.yaml, run-ids.txt, gap-list.md and a system sketch. The receipt records an S3 object presence check, a same-digest redownload check and a PASS secret scan. The participant-side receipt is traceable repository evidence, not an independently signed TM Forum receipt; the preserved final ZIP bytes were not fetched afresh for this research edition. In particular, the earlier 506-entry archive MUST NOT be identified as the final 152-file submission.

The preserved evidence and reviewer materials can be inspected without access to the discontinued AWS sandbox. The public TM Forum evidence hub at https://subodhkc.com/tm-forum links an offline forensic/judge HTML report, companion JSON evidence, final report, a judge-run portal, diagrams, technical CLI references, and a separate historical pre-event thesis. The repository-backed judge-run portal at https://subodhkc.com/tm-forum/TMF_JUDGE_RUN_PORTAL.html exposes named runs, commands, expected verdicts and evidentiary bounds. The HAIEC read-only judge workspace is linked at https://www.haiec.com/dashboard/assurance-lab; its registered VERIFY / VERIFY ALL and replay views are supplemental participant-operated interfaces over persisted evidence, not external scientific adjudication. An authenticated live end-to-end portal session was not independently executed as part of this paper's editorial verification.

The portal's original-system C16 results and later participant-system C7/C9 results remain explicitly distinct. Its separate October 6 post-submission A/B scoreboard reports scenario-grader comparisons under different run IDs; these are not the same as the initial archived S2 5/10 retest or an organizer-verified ISAF research benchmark. The original 9/10 C7 failure and later 10/10 remediation are not same-system/same-policy retests. The C9 observed breach and subsequent human/ServiceNow response are retrospective, not evidence of in-transaction prevention.

The AWS workshop environment is no longer accessible to the authors. This does not invalidate the retained empirical record: qualified HAIEC snapshots, original archive bytes, frozen policy/result

identities and preserved AWS-derived measurements support independently repeatable archived calculations and source-scoped assessments where the necessary inputs are available. Recomputing preserved values is distinct from obtaining a fresh AWS-signed source attestation or proving completeness of records that were never exported. The original source-primitive AWS replay and complete historical audit/event-origin attestation cannot be repeated from the retired sandbox; this is an explicitly disclosed archival limitation, not an outstanding publication action. The final paper claims source-scoped applied findings, archived deterministic calculations with specified reproduction bounds, and a participant-recorded successful organizer-script submission. It does not claim TM Forum endorsement, independent native-source attestation, independent peer review, production-wide generalization or complete Delegation/Effect closure. These limits are part of the finalized scientific record, not conditions that must be silently satisfied for an author-issued final edition.

Primary evidence pointers: participant submission receipt, https://github.com/subodhkc/haiec-website/blob/main/docs/handoff/TM%20FORUM/TM%20Forum%20Build%20Handoff/TMF_FINAL_SUBMISSION_RECEIPT_2026-10-06.md ; final snapshot, https://github.com/subodhkc/haiec-website/blob/main/docs/handoff/TM%20FORUM/TM%20Forum%20Build%20Handoff/TMF_2026_SUBMISSION_SNAPSHOT.json ; archived verifier run, https://github.com/subodhkc/tmf-2026-evidence/actions/runs/37909647446 ; hub, https://subodhkc.com/tm-forum .

52C. Archived Evidence Reproducibility and Contributor Technical Review Protocol

Archival evidentiary standing. The preserved HAIEC assessment snapshots, participant-captured AWS-derived telemetry, frozen policy definitions, run/evaluation identities, original evidence files, and deterministic result receipts constitute empirical research evidence within their stated source and time scope. An AWS workshop does not need to remain operational for a reader to validate archived bytes or independently recompute a preserved deterministic result. Conversely, data copied from a workshop is not itself an AWS-signed attestation of the native producer, completeness of unexported logs, or a new execution of the historical scenario. Source-qualified reproducibility and original-host attestation are different claims.

Four distinct verification levels apply. (1) Archive integrity: the original 506-file public manifest was checked on a fresh GitHub Actions checkout using the authors' read-only verification program; the CI runner provides a separate execution environment, not an unaffiliated scientific reviewer. (2) Archived analytical reproduction: inspect frozen rule, evidence denominator and named run, recompute original C16 token totals and available C9 duration arithmetic, and check preserved C7 event obligations and the S2 failed retest. (3) HAIEC persisted-result verification: the read-only judge workspace and the TM Forum judge portal expose source/policy/result references and a VERIFY workflow over retained HAIEC evidence. Independent reviewer success with authenticated live access must be reported by

that reviewer; the presence of a portal alone does not establish that a third party completed the replay. (4) Original AWS producer attestation: full platform-native historical log retrieval, independently established source completeness and organizer-controlled object signatures are outside this preserved record. The fourth limitation does not negate levels one through three.

Version and non-substitution rules are mandatory for every review. The original C16 same-policy 60,000-token comparison on system 4043efee is 35,559 SATISFIED versus 106,829 NOT_SATISFIED. The original C7 system 4043efee under ae6dda36 remains 9/10 NOT_SATISFIED; the later 44f861cf/dcfa6d38 10/10 SATISFIED result is a separate, system-scoped behavioral remediation. The later participant C9 44f861cf/13c07652 0/3 PASS and 1/3 BREACH (worst +286.026%) must not be attributed to the original 4043efee/346b policy. The original S2 5/10-to-5/10 outcome remains NOT_FIXED; the original DAI UNKNOWN remains UNKNOWN. The publicly verified 506-file predecessor archive is not the recorded final 152-file organizer-script submission ZIP. A synthetic canary, alert acknowledgment or ServiceNow incident cannot substitute for a production violation, pre-action business approval or verified downstream applied effect.

Requested contributor technical review. Kushal Gautam, identified elsewhere in this paper as a Data Engineering Advisor and participant in related TM Forum work, is an appropriate technical and reproducibility reviewer. Because of his prior collaboration, his assessment should be attributed as a contributor technical review, not described as independent external journal peer review. The requested review should critically examine: evidence hashes and source provenance; exact run/policy/result identity; arithmetic and event-set reproduction for C16/C9/C7; original versus remediation scope; handling of S2 and DAI negative results; frozen ISAF five-plane and orthogonal Delegation semantics; 78-checkpoint catalog claim limits; DCD v0.2/v0.3 benchmark claims; overlap or dependence among evidence producers; any missing material contradictory evidence; and privacy/licensing constraints on reviewer-facing artifacts. The original historical evidence, frozen results and submitted packages must remain unchanged.

Documenting the review. The reviewer should identify the exact paper edition and evidence commit/hash; scope of materials accessed; methods and commands used; calculations independently reproduced or not reproducible; material disagreements or limitations; suggested corrections; any relationship to the authors or implementation; and a dated signed conclusion. Reproducibility outcomes must be reported individually, not replaced by a blanket endorsement. The authors should document a point-by-point response and incorporate supported corrections without rewriting historical source or presenting a contributor review as publication-venue acceptance. The reviewer may use https://github.com/subodhkc/tmf-2026-evidence , https://subodhkc.com/tm-forum and, with separately authorized read-only access, https://www.haiec.com/dashboard/assurance-lab. The historical KUSHAL_REVIEW_CARD.md was written for the now-retired workshop and must not be interpreted as evidence that those AWS IDE commands remain executable.

Review-status disclosure. No written technical-review result, reviewer sign-off, independent external peer review, or scholarly acceptance is asserted in this edition. A dated review statement and author

response may be appended only after the reviewer actually supplies them, and any substantive revision should retain a distinct edition identity. The manuscript is a final author-issued research edition and a valid object for subsequent contributor and independent scholarly scrutiny, not a claimed peer-reviewed publication.

53. Standards and Regulatory Crosswalk: Informative, Not Equivalence

ISAF v2 includes an informative crosswalk because enterprises operate across multiple governance and compliance regimes.

The crosswalk must preserve one rule:

MAPPING != CONTROL_EFFECTIVENESS != COMPLIANCE.

53.1 NIST AI RMF

NIST AI RMF 1.0 is a voluntary, use-case-agnostic framework organized around GOVERN, MAP, MEASURE and MANAGE. NIST states that the framework is being revised (NIST AI RMF 1.0; NIST, 2026).

ISAF can provide technical evidence that supports activities inside those functions, for example evaluated scope, evidence lineage, control efficacy, monitoring and change, but ISAF does not replace organizational AI risk management.

53.2 NIST TEVV-Athlon

NIST's initial public draft of NIST AI 200-2, the TEVV-Athlon Framework for Evaluating AI Systems, presents a structured, extensible TEVV approach for assessing real-world impact and outcomes across AI applications, explicitly including agentic systems. NIST announced the draft on 7 August 2026 with public input open through 6 October 2026 (NIST AI 200-2 initial public draft, 2026).

ISAF is complementary: TEVV-Athlon provides broader evaluation methodology, while ISAF preserves claim-specific authority, capability, execution and consequence evidence boundaries. The draft is not treated as a finalized standard, certification mechanism, or source of ISAF delegation semantics.

53.3 NIST Agent Identity and Authorization

NIST NCCoE's 2026 Software and AI Agent Identity and Authorization effort has advanced from the February concept-paper stage into an active implementation-oriented project. NCCoE reports more than 600 responses and is standing up practical implementation work, including coordination with its DevSecOps program on agent identification, authentication, and authorization in the software-development lifecycle. The effort continues to focus on standards-based identification, authorization, auditing and non-repudiation for software and AI agents accessing tools, applications and data (NIST NCCoE, 2026).

ISAF complements this by separating identity and authorization evidence from delegated business decision rights, code capability and observed consequence.

53.4 ISO/IEC 42001

ISO/IEC 42001 specifies requirements for establishing, implementing, maintaining and continually improving an AI management system. ISO describes it as an organization-level management framework for AI risks and opportunities (ISO/IEC 42001:2023).

ISAF can support evidence, traceability and technical assurance inside an AI management system. It is not an ISO/IEC 42001 certification mechanism.

53.5 European Union AI Act

The EU AI Act became generally applicable on 2 August 2026, with staged exceptions. Following the 2026 AI Omnibus, Annex III high-risk rules apply from 2 December 2027 and high-risk systems embedded in regulated products from 2 August 2028. Commission materials identify high-risk obligations including risk assessment and mitigation, data quality, activity logging, technical documentation, deployer information, human oversight, robustness, cybersecurity and accuracy. Application still depends on legal role, classification, use case, timing and jurisdiction (Regulation (EU) 2024/1689; Regulation (EU) 2026/1744; European Commission AI Act Service Desk, 2026).

ISAF may support evidence for logging, traceability, oversight and technical risk-control documentation. It does not determine legal classification, satisfy conformity assessment by itself, or establish compliance.

53.6 OWASP Agent Control Standard

OWASP's GenAI Security Project published the Agent Control Standard (ACS) on 1 September 2026 as an open foundation for inspectable, traceable and instrumentable agents, standardized middleware hooks and declarative runtime-policy enforcement (OWASP GenAI Security Project, 2026).

ISAF is complementary: ACS can provide runtime control mechanisms and evidence; ISAF preserves whether the control was configured, evaluated, enforced and sufficient for the claim being made.

53.7 Model Context Protocol

The MCP 2026-07-28 specification introduced a stateless protocol core, header-based routing and authorization hardening including RFC 9207 issuer validation and issuer-bound client credentials, while formally deprecating Dynamic Client Registration in favor of Client ID Metadata Documents (Model Context Protocol, 2026).

ISAF consumes protocol authorization and tool relationships as relevant evidence while retaining the distinction between callable permission and delegated consequence-bearing decision rights.

53.8 OpenTelemetry GenAI conventions

OpenTelemetry's GenAI semantic conventions have moved to a dedicated GenAI semantic-conventions repository, where agent/tool conventions remain in Development status. They define interoperable operations including `invoke_agent`, `invoke_workflow`, `plan`, and `execute_tool`, making them useful as an observation substrate without making the telemetry authoritative for downstream business state (OpenTelemetry, 2026).

They do not make every span authoritative for downstream state or business consequence.

53.9 SLSA v1.2

At the publication-currency check, SLSA v1.2 was the approved SLSA specification and included Build and Source tracks. Its provenance model provides verifiable information about where, when and how software artifacts or source revisions were produced. ISAF can use that class of evidence for source/build identity, provenance and change, but SLSA provenance does not establish agent delegation, runtime execution or business consequence (SLSA v1.2).

54. What ISAF v2 Does Not Claim

The integrity of the framework depends on being explicit about its limits.

- ISAF does not prove that a probabilistic AI system will always behave correctly.
- ISAF does not make the underlying model deterministic; it can make bounded evaluation and evidence processing deterministic.
- ISAF does not require access to private chain-of-thought and does not claim to reconstruct internal cognition.
- ISAF does not infer human mental intent. It evaluates external authority and evidence artifacts.
- ISAF does not convert the presence of a control into proof of effectiveness.
- ISAF does not treat missing evidence as proof of safety or proof of violation.
- ISAF does not establish legal compliance or certification merely by mapping evidence to a standard.
- ISAF does not claim complete static recovery for arbitrary dynamic, reflective, distributed or opaque code.
- ISAF does not establish causality merely from timing, graph adjacency or correlation.
- ISAF does not claim the HAIEC product snapshot reviewed for this paper implements every research construct in the framework.
- ISAF v2 now has a completed frozen DCD-Bench v0.2 reference-suite result. On the 20 preregistered synthetic/reference fixtures, the independent reference evaluator reproduced all expected epistemic outcomes, preserved the preregistered abstentions, produced zero registered hard-negative violations, and replayed deterministically across two sealed runs. A subsequent post-hoc adversarial challenge of the unchanged evaluator preserved the 23/23 qualification baseline but failed 8/8 broader boundary challenges, exposing implementation-generalization gaps in parameter completeness, unknown-operation handling, path-bound mediation and observation, control-flow-sensitive reachability, sample provenance, and common JavaScript callable forms. Those eight published gaps were then repaired only in a separately versioned hardened v0.3 evaluator. At commit c8332e806bd9f99bdd2490c98342ddfde8f660f8, the hardened evaluator re-passed the frozen v0.2 23/23 qualification baseline, passed 34/34 existing hardening tests including all eight red-team regression properties, and preserved all 20 frozen v0.2 epistemic outcomes; no held-out cases or held-out gold were used. This establishes hardened reference-evaluator qualification, not held-out, production, or cross-program accuracy. The product DAI assembler remains outside benchmark truth unless independently brought into conformance.
- ISAF does not claim generalized trajectory-safety proof or universal recovery proof.

55. Current Research Frontiers

Several high-value areas should remain explicit research frontiers rather than being hidden from the paper:

- Generic Critical Action Contract derivation and completeness.
- Full protected-commitment residual computation across heterogeneous actions.
- MinResolve / Q*: determining the minimum additional trusted fact, approval or evidence needed to close a consequential ambiguity.
- Trajectory and composition safety across repeated or concurrent actions.
- Causal influence reconstruction across RAG, memory and shared state.
- Runtime-principal binding across distributed agent systems.
- Recovery evidence and service-restoration proof.
- Evaluation integrity under reward hacking, grader awareness and evidence manipulation.
- Portable delegation attenuation across MCP and agent-to-agent chains.

These are opportunities for future ISAF versions and benchmark extensions, not defects to conceal.

56. Recommended Assurance Output

The full technical evidence may contain thousands of relationships and artifacts. The assurance summary should be smaller.

A concise assurance view should answer:

- What system and business process were evaluated?
- Who owns the system and consequential outcome?
- What are the highest-consequence autonomous actions?
- Which decision rights are established as delegated?
- Where are policy, effective grants and code capability materially misaligned?
- Which controls are only configured versus evidenced as enforced?
- What materially changed since the prior evaluation?
- What are the largest unresolved Proof Frontiers?
- What evidence or action is required next?

The assurance summary should not hide uncertainty. It should compress complexity without compressing truth.

57. The Final Paradox

The more capable an AI agent becomes, the easier it is to measure the wrong thing.

We can count tools.

We can count permissions.

We can count calls.

We can count traces.

We can count controls.

Every number can increase while accountability decreases.

An agent with ten tools may be safer than one with two if its decision rights are explicit, its scope is bounded, its controls are enforced and its effects are evidenced. An agent with perfect logs may still be ungoverned if nobody can establish who delegated the consequential choice.

The central problem is therefore not insufficient data.

It is false equivalence.

Permission is treated as delegation. Registration is treated as reachability. Reachability is treated as execution. Execution is treated as effect. Effect is treated as consequence. A configured control is treated as enforced. A log is treated as proof. A current fact is treated as historical truth.

ISAF v2 exists to keep those things apart long enough to understand how they actually fit together.

58. Conclusion: Accountability Must Reach the Consequence

AI systems are becoming actors inside the enterprise.

They read private data, call tools, change infrastructure, move money, modify access, communicate externally, coordinate with other agents, and make decisions at speeds conventional governance was not designed to follow.

The answer cannot be to place a human click in front of every action. Nor can it be to declare autonomy safe because identity, permission, policy, observability and guardrails exist somewhere in the architecture.

The more scalable answer is to make authority and evidence precise.

Know what shaped the decision.

Know what the system can reach.

Know what policy permits.

Know what access is actually granted.

Know which choices were delegated.

Know what the code can cause.

Know what runtime evidence establishes.

Know what effect occurred.

Know which controls actually mediated it.

Know what changed.

And when the evidence stops, say that it stops.

That is not weaker assurance.

It is the foundation of stronger assurance.

High-assurance autonomy does not begin when every dashboard turns green. It begins when the organization can explain what each green light actually proves.

ISAF v1 traced the instructions that shaped an AI system.

ISAF v2 follows those instructions until they become authority, action and consequence.

Accountability must reach the consequence.

References

South, T., Marro, S., Hardjono, T., Mahari, R., Whitney, C. D., Chan, A., & Pentland, A. (2025). Position: AI Agents Need Authenticated Delegation. Proceedings of the 42nd International Conference on Machine Learning, PMLR 267.

Lodderstedt, T., Richer, J., & Campbell, B. (2023). RFC 9396: OAuth 2.0 Rich Authorization Requests. IETF / RFC Editor.

NIST NCCoE. (2026). Software and AI Agent Identity and Authorization. Concept paper, project page, and September 29, 2026 implementation-use-case update. National Cybersecurity Center of Excellence.

Shi, T., He, J., Wang, Z., Wu, L., Li, H., Guo, W., & Song, D. (2025). Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703.

Li, H., Wen, R., Shi, S., Zhang, N., & Xiao, C. (2026). AgentDyn: A Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks of Real-World Agent Security System. arXiv:2602.03117.

Xu, Z., Zhang, X., Luo, H., Jin, Y., Dong, Y., & Salam, H. (2026). MasDrift: Benchmarking Authorization Preservation Across Multi-Agent Architectures. arXiv:2608.07556.

KC, S. (2025). The Instruction Stack Audit Framework (ISAF): A Technical Methodology for Tracing AI Accountability Across Nine Abstraction Layers. Zenodo. DOI 10.5281/zenodo.18080355.

Tabassi, E. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1. National Institute of Standards and Technology. DOI 10.6028/NIST.AI.100-1.

NIST. (2026). AI Risk Management Framework. National Institute of Standards and Technology. AI RMF 1.0 revision status and resources.

NIST. (2026). The TEVV-Athlon Framework for Evaluating AI Systems. NIST AI 200-2, Initial Public Draft. DOI 10.6028/NIST.AI.200-2.ipd. Public input announced through 6 October 2026.

ISO/IEC. (2023). ISO/IEC 42001:2023, Information technology: Artificial intelligence: Management system. Edition 1.

European Parliament & Council of the European Union. (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). OJ L 2024/1689; ELI: data.europa.eu/eli/reg/2024/1689/oj.

European Parliament & Council of the European Union. (2026). Regulation (EU) 2026/1744 amending Regulation (EU) 2024/1689 as regards simplification of the implementation of harmonised rules on artificial intelligence (Digital Omnibus on AI). OJ L 2026/1744; ELI: data.europa.eu/eli/reg/2026/1744/oj.

European Commission AI Act Service Desk. (2026). EU AI Act implementation timeline and enforcement guidance, including staged high-risk application dates.

OWASP GenAI Security Project. (2026). Agent Control Standard (ACS). Published 1 September 2026.

Model Context Protocol. (2026). Model Context Protocol Specification, revision 2026-07-28.

OpenTelemetry. (2026). Semantic Conventions v1.44.0; Generative AI and agent/tool semantic conventions. Relevant conventions remain in Development status.

MunichTech EXPO / Devpost. (2026). MunichTech EXPO Hackathon 2026: Grand Challenge definition, requirements and judging criteria.

HAIEC. (2026). AI Action Path Assurance | Deterministic Assurance: MunichTech EXPO Devpost submission.

HAIEC. (2026). Kestrel frozen MunichTech evidence package: evidence manifest, public manifest, and README; source commit 5e65843fddfe5f907485b798e464ad37b3b3b2c7.

HAIEC. (2026). Batch F: Joern Qualification Benchmark Report. Joern v4.0.610 / pysrc2cpg; 17 contract fixtures plus Kestrel Voice slice; committed qualification artifacts.

ISAF v2.0. (2026). HAIEC MCP Reference Implementation Boundary Record v1.3: repo-grounded implementation note for agent-operated deterministic Assurance; refreshed against Main df24a8d6b2d97a433ae41a9670c475bc09e60ee4; full product conformance and production fresh-run E2E qualification are not claimed.

ISAF v2.0. (2026). Normative Checkpoint Catalog Freeze v1.0: 78 core checkpoints across 20 claim-owner families.

ISAF v2.0. (2026). Producer Qualification Protocol Freeze v1.0: claim-class-specific evidence-producer qualification methodology.

DCD-Bench v0.2. (2026). Frozen Fixture, Gold-Label & Execution Specification plus Sealed Empirical Results: B01–B20 reference-suite conformance with deterministic twin-run replay; post-hoc adversarial limitations are published separately and explicitly block production-generalization claims.

DCD Evaluator v0.3 Hardening Qualification. (2026). Qualified commit c8332e806bd9f99bdd2490c98342ddfde8f660f8; GitHub Actions run 36814726049; frozen v0.2 baseline 23/23 PASS; v0.3 hardening suite 34/34 PASS; all 20 frozen v0.2 epistemic outcomes preserved; no held-out cases or gold used. Sealed artifact 11140763317; SHA-256 a2363ae1a6966fb1aab6dac59a5515410a75163ccb8fbd0d1dbbc4d921f98b3a.

KC, S. (2026). When Every Control Says Yes: Beyond Authorization: Measuring Delegation-Capability Divergence in Agentic AI Systems. HAIEC Research Artifact HAIEC-RA-DCD-2026-001, v0.1. DOI pending; first-party research framework/protocol, not peer reviewed.

MunichTech EXPO. (2026). Official Grand Challenge recognition package: certificate/verification record, awards listing, newsroom announcement, and published score record.

TM Forum. (2026). Trustworthy AI & Data Hackathon: Agentic Assurance – The Quest for Proof, Innovate Americas 2026: official hackathon program and team acceptance correspondence.

HAIEC Research. (2026). TM Forum GitHub Archive Verification and Reproduction Record v1.0: SHA-256/GZIP/TAR verification, normalized control arithmetic and unresolved organizer submission boundary. https://docs.google.com/document/d/1sDs-Aqz17IIVZKa8LpQsp2Wypk51zzluFoRLWHsHVhA/edit

HAIEC Research. (2026). TM Forum 2026 Agentic Assurance Evidence Pack, v7a0bb5f3 [Data set and evidence archive]. GitHub. https://github.com/subodhkc/tmf-2026-evidence ; immutable version: https://github.com/subodhkc/tmf-2026-evidence/releases/tag/v7a0bb5f3.

HAIEC Research. (2026). Public TM Forum Evidence Repository Release Verification v1.0. https://docs.google.com/document/d/1PWKHuLWwUkGzRzDjUQpGoXWCin8xsENupcZ0dQmMLZk/edit.

HAIEC Research. (2026). ISAF v2: Enterprise Evidence Case 001: TM Forum 2026. Consolidated applied-study annex v1.0; participant final submission receipt now documented, with external primitive source-attestation limits retained. https://docs.google.com/document/d/1S3F0d6xB0N5wvMApRiGgKiEsqI6Aw0f0UDIxDRGPo5M/edit

HAIEC Research. (2026). ISAF v2: Operational Assurance Process and Control Test Profile. Informative working draft v1.0; not part of the normative 78-checkpoint contract. https://docs.google.com/document/d/1h9mes5RDIj2mNcoOb2mOu3G2-PJWDYbWsbanXWv2af0/edit

Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. Advances in Neural Information Processing Systems 37. DOI 10.52202/079017-2636.

Zhang, H., Huang, J., Mei, K., Yao, Y., Wang, Z., Zhan, C., Wang, H., & Zhang, Y. (2025). Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. International Conference on Learning Representations (ICLR 2025). arXiv:2410.02644.

Zhu, G., & Wang, C. (2026). Intent-Governed Tool Authorization for AI Agents. arXiv:2606.22916.

Parisel, C. (2026). A Theory of Least Autonomy in AI. arXiv:2607.09744.

Dantuluri, P. S. V., & Sundi, J. (2026). Delegation Without Trust: An Empirical Gap Analysis of Identity, Authorization, and Runtime Governance in Multi-Agent LLM Systems. arXiv:2609.00267.

SLSA. SLSA Specification v1.2. Status: Approved; Build and Source tracks.

in-toto. in-toto Specification v1.0; in-toto Attestation Framework (current release line v1.2.0 at the publication-currency check).

Korn Ferry. (2026). Ready or Not: 2nd Annual Board and CEO Risk Study.

PwC. (2026). Building Trust in Agentic AI: Three Governance Shifts.

Spencer Stuart. (2026). Nominating and Governance Committee Chair Priorities: 2026 and Beyond.